```
package com.dxkj.myshell.crypto
import android.util.Base64
import java.security.KeyStore
import javax.crypto.Cipher
import javax.crypto.KeyGenerator
import javax.crypto.SecretKey
import javax.crypto.spec.GCMParameterSpec
object CryptoManager {
    private const val ANDROID_KEYSTORE = "AndroidKeyStore"
    private const val KEY_ALIAS = "[redacted]"
    private const val TRANSFORMATION = "AES/GCM/NoPadding"
    private const val IV_SIZE = 12
    private const val TAG_BITS = 128
    fun encryptToBase64(plaintext: String): String {
        val key = getOrCreateKey()
        val cipher = Cipher.getInstance(TRANSFORMATION)
        cipher.init(Cipher.ENCRYPT_MODE, key)
        val iv = cipher.iv
        val encrypted = cipher.doFinal(plaintext.toByteArray(Charsets.UTF_8))
        val combined = ByteArray(iv.size + encrypted.size)
        System.arraycopy(iv, 0, combined, 0, iv.size)
        System.arraycopy(encrypted, 0, combined, iv.size, encrypted.size)
        return Base64.encodeToString(combined, Base64.NO_WRAP)
    }
    fun decryptFromBase64(base64: String?): String? {
        if (base64.isNullOrBlank()) return null
        val combined = try {
            Base64.decode(base64, Base64.NO_WRAP)
        } catch (_: Throwable) {
            return null
        }
        if (combined.size <= IV_SIZE) return null
        val iv = combined.copyOfRange(0, IV_SIZE)
        val ciphertext = combined.copyOfRange(IV_SIZE, combined.size)
        val key = getOrCreateKey()
        val cipher = Cipher.getInstance(TRANSFORMATION)
        cipher.init(Cipher.DECRYPT_MODE, key, GCMParameterSpec(TAG_BITS, iv))
        val plain = cipher.doFinal(ciphertext)
        return plain.toString(Charsets.UTF_8)
    }
    private fun getOrCreateKey(): SecretKey {
        val ks = KeyStore.getInstance(ANDROID_KEYSTORE).apply { load(null) }
        val existing = (ks.getEntry(KEY_ALIAS, null) as? KeyStore.SecretKeyEntry)?.secretKey
        if (existing != null) return existing
        val kg = KeyGenerator.getInstance("AES", ANDROID_KEYSTORE)
        val spec = android.security.keystore.KeyGenParameterSpec.Builder(
            KEY_ALIAS,
            android.security.keystore.KeyProperties.PURPOSE_ENCRYPT or android.security.keystore.KeyProperties.PURP
        )
            .setBlockModes(android.security.keystore.KeyProperties.BLOCK_MODE_GCM)
```

```
            .setEncryptionPaddings(android.security.keystore.KeyProperties.ENCRYPTION_PADDING_NONE)
            .setRandomizedEncryptionRequired(true)
            .build()
        kg.init(spec)
        return kg.generateKey()
    }
}
package com.dxkj.myshell.data.db
import androidx.room.Database
import androidx.room.RoomDatabase
@Database(
    entities = [
        HostEntity::class,
        KeyEntity::class,
    ],
    version = 3,
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun hostDao(): HostDao
    abstract fun keyDao(): KeyDao
}
package com.dxkj.myshell.data.db
import android.content.Context
import androidx.room.Room
object DbProvider {
    @Volatile
    private var instance: AppDatabase? = null
    fun get(context: Context): AppDatabase {
        val existing = instance
        if (existing != null) return existing
        return synchronized(this) {
            val again = instance
            if (again != null) {
                again
            } else {
                val created = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "myshell.db",
                )
                    .fallbackToDestructiveMigration()
                    .build()
                instance = created
                created
            }
        }
    }
}
package com.dxkj.myshell.data.db
import androidx.room.Entity
```

```
import androidx.room.PrimaryKey
@Entity(tableName = "hosts")
data class HostEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val host: String,
    val port: Int,
    val username: String,
    val authType: String,
    val passwordEnc: String?,
    val privateKeyId: Long?,
    val createdAtEpochMs: Long,
    val updatedAtEpochMs: Long,
)
@Entity(tableName = "keys")
data class KeyEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val privateKeyPemEnc: String,
    val passphraseEnc: String?,
    val createdAtEpochMs: Long,
    val updatedAtEpochMs: Long,
)
package com.dxkj.myshell.data.db
import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow
@Dao
interface HostDao {
    @Query("SELECT * FROM hosts ORDER BY updatedAtEpochMs DESC")
    fun observeAll(): Flow<List<HostEntity>>
    @Query("SELECT * FROM hosts WHERE id = :id LIMIT 1")
    suspend fun getById(id: Long): HostEntity?
    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insert(entity: HostEntity): Long
    @Update
    suspend fun update(entity: HostEntity)
    @Delete
    suspend fun delete(entity: HostEntity)
    @Query("DELETE FROM hosts WHERE id = :id")
    suspend fun deleteById(id: Long)
}
package com.dxkj.myshell.data.db
import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
```

```
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow
@Dao
interface KeyDao {
    @Query("SELECT * FROM keys ORDER BY updatedAtEpochMs DESC")
    fun observeAll(): Flow<List<KeyEntity>>
    @Query("SELECT * FROM keys WHERE id = :id LIMIT 1")
    suspend fun getById(id: Long): KeyEntity?
    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insert(entity: KeyEntity): Long
    @Update
    suspend fun update(entity: KeyEntity)
    @Delete
    suspend fun delete(entity: KeyEntity)
    @Query("DELETE FROM keys WHERE id = :id")
    suspend fun deleteById(id: Long)
}
package com.dxkj.myshell.data.prefs
import android.app.Application
import android.content.Context
import androidx.compose.ui.graphics.Color
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
object AppPreferences {
    private const val PREFS = "myshell_app_settings"
    private const val KEY_THEME = "theme"
    private const val KEY_TERMINAL_COLOR_SCHEME = "terminal_color_scheme"
    private const val KEY_TERMINAL_FONT_SIZE = "terminal_font_size"
    private const val KEY_TERMINAL_COPY_ON_SELECT = "terminal_copy_on_select"
    const val DEFAULT_FONT_SIZE = 14
    const val MIN_FONT_SIZE = 8
    const val MAX_FONT_SIZE = 32
    private lateinit var prefs: android.content.SharedPreferences
    private val _themeMode = MutableStateFlow(ThemeMode.SYSTEM)
    val themeMode: StateFlow<ThemeMode> = _themeMode.asStateFlow()
    private val _terminalColorScheme = MutableStateFlow(TerminalColorScheme.DEFAULT)
    val terminalColorScheme: StateFlow<TerminalColorScheme> = _terminalColorScheme.asStateFlow()
    private val _terminalFontSize = MutableStateFlow(DEFAULT_FONT_SIZE)
    val terminalFontSize: StateFlow<Int> = _terminalFontSize.asStateFlow()
    private val _terminalCopyOnSelect = MutableStateFlow(true)
    val terminalCopyOnSelect: StateFlow<Boolean> = _terminalCopyOnSelect.asStateFlow()
    fun init(application: Application) {
        if (::prefs.isInitialized) return
        prefs = application.getSharedPreferences(PREFS, Context.MODE_PRIVATE)
        _themeMode.value = ThemeMode.fromString(prefs.getString(KEY_THEME, null))
        _terminalColorScheme.value = TerminalColorScheme.fromString(prefs.getString(KEY_TERMINAL_COLOR_SCHEME, null
        _terminalFontSize.value = prefs.getInt(KEY_TERMINAL_FONT_SIZE, DEFAULT_FONT_SIZE)
```

```
            .coerceIn(MIN_FONT_SIZE, MAX_FONT_SIZE)
        _terminalCopyOnSelect.value = prefs.getBoolean(KEY_TERMINAL_COPY_ON_SELECT, true)
    }
    fun setThemeMode(mode: ThemeMode) {
        if (!::prefs.isInitialized) return
        prefs.edit().putString(KEY_THEME, mode.name).apply()
        _themeMode.value = mode
    }
    fun setTerminalColorScheme(scheme: TerminalColorScheme) {
        if (!::prefs.isInitialized) return
        prefs.edit().putString(KEY_TERMINAL_COLOR_SCHEME, scheme.name).apply()
        _terminalColorScheme.value = scheme
    }
    fun setTerminalFontSize(sizeSp: Int) {
        if (!::prefs.isInitialized) return
        val v = sizeSp.coerceIn(MIN_FONT_SIZE, MAX_FONT_SIZE)
        prefs.edit().putInt(KEY_TERMINAL_FONT_SIZE, v).apply()
        _terminalFontSize.value = v
    }
    fun setTerminalCopyOnSelect(enabled: Boolean) {
        if (!::prefs.isInitialized) return
        prefs.edit().putBoolean(KEY_TERMINAL_COPY_ON_SELECT, enabled).apply()
        _terminalCopyOnSelect.value = enabled
    }
    fun argbLongToColor(argb: Long): Color = Color(argb and 0xFFFFFFFFL)
    fun defaultTerminalColors(): Pair<Color, Color> {
        val s = if (::prefs.isInitialized) _terminalColorScheme.value else TerminalColorScheme.DEFAULT
        return Pair(argbLongToColor(s.foreground), argbLongToColor(s.background))
    }
    enum class ThemeMode(val label: String) {
        SYSTEM(" 跟随系统"),
        LIGHT(" 浅色"),
        DARK(" 深色");
        companion object {
            fun fromString(value: String?): ThemeMode =
                entries.find { it.name == value } ?: SYSTEM
        }
    }
    enum class TerminalColorScheme(
        val label: String,
        val background: Long,
        val foreground: Long,
    ) {
        CLASSIC_GREEN(" 经典绿", 0xFF000000, 0xFF00FF00),
        LIGHT(" 浅色", 0xFFFFFFFF, 0xFF1A1A1A),
        SOLARIZED_DARK("Solarized Dark", 0xFF002B36, 0xFF839496),
        DRACULA("Dracula", 0xFF282A36, 0xFFF8F8F2),
        MONOKAI("Monokai", 0xFF272822, 0xFFF8F8F2),
        NORD("Nord", 0xFF2E3440, 0xFFD8DEE9),
        GRUVBOX("Gruvbox", 0xFF282828, 0xFFEBDBB2),
```

```
        TOKYO_NIGHT("Tokyo Night", 0xFF1A1B26, 0xFFA9B1D6),
        QBASIC("QBasic", 0xFF0000AA, 0xFFAAAAAA),
        AMBER(" 琥珀", 0xFF1A1000, 0xFFFFB000),
        PINK(" 粉色", 0xFF2D001E, 0xFFFF9EC6),
        LAVENDER(" 薰衣草", 0xFF1E1629, 0xFFCDB4DB),
        OCEAN(" 海洋", 0xFF0A192F, 0xFF64FFDA);
        companion object {
            val DEFAULT: TerminalColorScheme = CLASSIC_GREEN
            fun fromString(value: String?): TerminalColorScheme {
                if (value.isNullOrBlank() || value == "HAVEN") return DEFAULT
                return entries.find { it.name == value } ?: DEFAULT
            }
        }
    }
}
package com.dxkj.myshell.data.repo
import com.dxkj.myshell.data.db.HostDao
import com.dxkj.myshell.data.db.HostEntity
import com.dxkj.myshell.crypto.CryptoManager
import kotlinx.coroutines.flow.Flow
class HostRepository(
    private val dao: HostDao,
) {
    fun observeAll(): Flow<List<HostEntity>> = dao.observeAll()
    suspend fun getById(id: Long): HostEntity? = dao.getById(id)
    suspend fun upsert(
        id: Long?,
        name: String,
        host: String,
        port: Int,
        username: String,
        authType: String,
        password: String?,
        privateKeyId: Long?,
        nowEpochMs: Long,
    ): Long {
        val cleanName = name.trim()
        val cleanHost = host.trim()
        val cleanUsername = username.trim()
        require(cleanName.isNotEmpty()) { "name required" }
        require(cleanHost.isNotEmpty()) { "host required" }
        require(port in 1..65535) { "port invalid" }
        require(cleanUsername.isNotEmpty()) { "username required" }
        require(authType == "password" || authType == "key") { "authType invalid" }
        if (authType == "password") require(!password.isNullOrBlank()) { "password required" }
        if (authType == "key") require(privateKeyId != null) { "privateKeyId required" }
        val passwordEnc = if (authType == "password") CryptoManager.encryptToBase64(password!!) else null
        return if (id == null) {
            dao.insert(
                HostEntity(
```

```
                    name = cleanName,
                    host = cleanHost,
                    port = port,
                    username = cleanUsername,
                    authType = authType,
                    passwordEnc = passwordEnc,
                    privateKeyId = if (authType == "key") privateKeyId else null,
                    createdAtEpochMs = nowEpochMs,
                    updatedAtEpochMs = nowEpochMs,
                ),
            )
        } else {
            val existing = dao.getById(id) ?: error("host not found")
            dao.update(
                existing.copy(
                    name = cleanName,
                    host = cleanHost,
                    port = port,
                    username = cleanUsername,
                    authType = authType,
                    passwordEnc = passwordEnc,
                    privateKeyId = if (authType == "key") privateKeyId else null,
                    updatedAtEpochMs = nowEpochMs,
                ),
            )
            id
        }
    }
    suspend fun deleteById(id: Long) {
        dao.deleteById(id)
    }
}
package com.dxkj.myshell.data.repo
import com.dxkj.myshell.data.db.KeyDao
import com.dxkj.myshell.data.db.KeyEntity
import com.dxkj.myshell.crypto.CryptoManager
import kotlinx.coroutines.flow.Flow
class KeyRepository(
    private val dao: KeyDao,
) {
    fun observeAll(): Flow<List<KeyEntity>> = dao.observeAll()
    suspend fun getById(id: Long): KeyEntity? = dao.getById(id)
    suspend fun getDecryptedById(id: Long): DecryptedKey? {
        val e = dao.getById(id) ?: return null
        val pem = CryptoManager.decryptFromBase64(e.privateKeyPemEnc) ?: return null
        val pass = CryptoManager.decryptFromBase64(e.passphraseEnc)
        return DecryptedKey(
            id = e.id,
            name = e.name,
            privateKeyPem = pem,
```

```
            passphrase = pass,
        )
    }
    suspend fun insert(
        name: String,
        privateKeyPem: String,
        passphrase: String?,
        nowEpochMs: Long,
    ): Long {
        val cleanName = name.trim()
        require(cleanName.isNotEmpty()) { "name required" }
        require(privateKeyPem.contains("PRIVATE KEY")) { "not a private key" }
        val pemEnc = CryptoManager.encryptToBase64(privateKeyPem)
        val passEnc = passphrase?.takeIf { it.isNotBlank() }?.let { CryptoManager.encryptToBase64(it) }
        return dao.insert(
            KeyEntity(
                name = cleanName,
                privateKeyPemEnc = pemEnc,
                passphraseEnc = passEnc,
                createdAtEpochMs = nowEpochMs,
                updatedAtEpochMs = nowEpochMs,
            ),
        )
    }
    suspend fun deleteById(id: Long) {
        dao.deleteById(id)
    }
}
data class DecryptedKey(
    val id: Long,
    val name: String,
    val privateKeyPem: String,
    val passphrase: String?,
)
package com.dxkj.myshell
import android.os.Bundle
import androidx.activity.ComponentActivity
import com.dxkj.myshell.terminal.TerminalSessionPool
import androidx.activity.compose.setContent
import androidx.appcompat.app.AppCompatDelegate
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import com.dxkj.myshell.data.prefs.AppPreferences
```

```
import com.dxkj.myshell.ui.AppNav
import com.dxkj.myshell.ui.theme.MyShellTheme
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        AppPreferences.init(application)
        setContent {
            AppRoot()
        }
    }
    override fun onResume() {
        super.onResume()
        TerminalSessionPool.init(application)
        TerminalSessionPool.onApplicationResume()
    }
}
@Composable
private fun AppRoot() {
    val themeMode by AppPreferences.themeMode.collectAsState()
    LaunchedEffect(themeMode) {
        AppCompatDelegate.setDefaultNightMode(
            when (themeMode) {
                AppPreferences.ThemeMode.LIGHT -> AppCompatDelegate.MODE_NIGHT_NO
                AppPreferences.ThemeMode.DARK -> AppCompatDelegate.MODE_NIGHT_YES
                AppPreferences.ThemeMode.SYSTEM -> AppCompatDelegate.MODE_NIGHT_FOLLOW_SYSTEM
            },
        )
    }
    val darkTheme = when (themeMode) {
        AppPreferences.ThemeMode.LIGHT -> false
        AppPreferences.ThemeMode.DARK -> true
        AppPreferences.ThemeMode.SYSTEM -> isSystemInDarkTheme()
    }
    MyShellTheme(darkTheme = darkTheme) {
        Surface(modifier = Modifier.fillMaxSize(), color = MaterialTheme.colorScheme.background) {
            AppNav()
        }
    }
}
package com.dxkj.myshell
import android.app.Application
import android.app.NotificationChannel
import android.app.NotificationManager
import android.os.Build
import org.bouncycastle.jce.provider.BouncyCastleProvider
import java.security.Security
class MyShellApp : Application() {
    override fun onCreate() {
        super.onCreate()
        try {
```

```
            Security.removeProvider("BC")
            Security.insertProviderAt(BouncyCastleProvider(), 1)
        } catch (_: Throwable) {
        }
        if (Build.VERSION.SDK_INT >= 26) {
            try {
                val nm = getSystemService(NotificationManager::class.java)
                nm.createNotificationChannel(
                    NotificationChannel(
                        "transfer",
                        " 文件传输",
                        NotificationManager.IMPORTANCE_LOW,
                    ),
                )
                nm.createNotificationChannel(
                    NotificationChannel(
                        PortForwardHoldService.CHANNEL_ID,
                        " 后台保持连接",
                        NotificationManager.IMPORTANCE_LOW,
                    ),
                )
            } catch (_: Throwable) {
            }
        }
    }
}
package com.dxkj.myshell
import android.app.Notification
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.os.Build
import android.os.IBinder
import androidx.core.app.NotificationCompat
import androidx.core.content.ContextCompat
class PortForwardHoldService : Service() {
    override fun onBind(intent: Intent?): IBinder? = null
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        val forwards = intent?.getIntExtra(EXTRA_FORWARD_COUNT, 0) ?: 0
        val sshSessions = intent?.getIntExtra(EXTRA_SSH_SESSION_COUNT, 0) ?: 0
        if (forwards <= 0 && sshSessions <= 0) {
            if (Build.VERSION.SDK_INT >= 24) {
                stopForeground(STOP_FOREGROUND_REMOVE)
            } else {
                @Suppress("DEPRECATION")
                stopForeground(true)
            }
            stopSelf()
            return START_NOT_STICKY
```

```
        }
        startForeground(NOTIFICATION_ID, buildNotification(this, forwards, sshSessions))
        return START_STICKY
    }
    companion object {
        const val EXTRA_FORWARD_COUNT = "active_forward_count"
        const val EXTRA_SSH_SESSION_COUNT = "connected_ssh_session_count"
        private const val NOTIFICATION_ID = 7102
        fun update(
            context: Context,
            activeForwardCount: Int,
            connectedSshSessionCount: Int,
        ) {
            val app = context.applicationContext
            val i = Intent(app, PortForwardHoldService::class.java)
                .putExtra(EXTRA_FORWARD_COUNT, activeForwardCount)
                .putExtra(EXTRA_SSH_SESSION_COUNT, connectedSshSessionCount)
            if (activeForwardCount > 0 || connectedSshSessionCount > 0) {
                ContextCompat.startForegroundService(app, i)
            } else {
                try {
                    app.stopService(Intent(app, PortForwardHoldService::class.java))
                } catch (_: Throwable) {
                }
            }
        }
        private fun buildNotification(
            ctx: Context,
            forwardCount: Int,
            sshSessionCount: Int,
        ): Notification {
            val open = PendingIntent.getActivity(
                ctx,
                0,
                Intent(ctx, MainActivity::class.java).addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP),
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
            )
            val parts = mutableListOf<String>()
            if (sshSessionCount > 0) {
                parts += "${sshSessionCount} 个 SSH 会话"
            }
            if (forwardCount > 0) {
                parts += "${forwardCount} 条本地端口转发"
            }
            val text = if (parts.isEmpty()) " 点按返回应用" else parts.joinToString("; ") + "; 点按返回应用"
            return NotificationCompat.Builder(ctx, CHANNEL_ID)
                .setSmallIcon(android.R.drawable.ic_menu_share)
                .setContentTitle("MyShell 后台保持连接")
                .setContentText(text)
                .setContentIntent(open)
```

```
                .setOngoing(true)
                .setOnlyAlertOnce(true)
                .setPriority(NotificationCompat.PRIORITY_LOW)
                .build()
        }
        internal const val CHANNEL_ID = "port_forward_hold"
    }
}
package com.dxkj.myshell.sftp
import net.schmizz.sshj.xfer.LocalDestFile
import net.schmizz.sshj.xfer.LocalFileFilter
import net.schmizz.sshj.xfer.LocalSourceFile
import java.io.IOException
import java.io.InputStream
import java.io.OutputStream
class CountingOutputStream(
    private val delegate: OutputStream,
    private val onBytes: (Long) -> Unit,
) : OutputStream() {
    private var count = 0L
    override fun write(b: Int) {
        delegate.write(b)
        count += 1
        onBytes(count)
    }
    override fun write(b: ByteArray, off: Int, len: Int) {
        delegate.write(b, off, len)
        count += len.toLong()
        onBytes(count)
    }
    override fun flush() = delegate.flush()
    override fun close() = delegate.close()
}
class CountingInputStream(
    private val delegate: InputStream,
    private val onBytes: (Long) -> Unit,
) : InputStream() {
    private var count = 0L
    override fun read(): Int {
        val r = delegate.read()
        if (r >= 0) {
            count += 1
            onBytes(count)
        }
        return r
    }
    override fun read(b: ByteArray, off: Int, len: Int): Int {
        val n = delegate.read(b, off, len)
        if (n > 0) {
            count += n.toLong()
```

```
            onBytes(count)
        }
        return n
    }
    override fun close() = delegate.close()
}
class OutputStreamDestFile(
    private val name: String,
    private val length: Long,
    private val open: (append: Boolean) -> OutputStream,
    private val onProgress: (Long) -> Unit,
) : LocalDestFile {
    override fun getLength(): Long = length
    @Throws(IOException::class)
    override fun getOutputStream(): OutputStream = getOutputStream(false)
    @Throws(IOException::class)
    override fun getOutputStream(append: Boolean): OutputStream {
        return CountingOutputStream(open(append), onProgress)
    }
    override fun getChild(child: String): LocalDestFile = getTargetFile(child)
    override fun getTargetFile(name: String): LocalDestFile =
        OutputStreamDestFile(
            name = name,
            length = length,
            open = open,
            onProgress = onProgress,
        )
    override fun getTargetDirectory(name: String): LocalDestFile = this
    override fun setPermissions(perms: Int) {}
    override fun setLastAccessedTime(time: Long) {}
    override fun setLastModifiedTime(time: Long) {}
}
class InputStreamSourceFile(
    private val name: String,
    private val length: Long,
    private val open: () -> InputStream,
    private val onProgress: (Long) -> Unit,
) : LocalSourceFile {
    override fun getName(): String = name
    override fun getLength(): Long = length
    @Throws(IOException::class)
    override fun getInputStream(): InputStream = CountingInputStream(open(), onProgress)
    override fun getPermissions(): Int = 420
    override fun isFile(): Boolean = true
    override fun isDirectory(): Boolean = false
    override fun getChildren(filter: LocalFileFilter): Iterable<LocalSourceFile> = emptyList()
    override fun providesAtimeMtime(): Boolean = false
    override fun getLastAccessTime(): Long = 0
    override fun getLastModifiedTime(): Long = 0
}
```

```
package com.dxkj.myshell.sftp
import android.content.ContentResolver
import android.content.ContentValues
import android.content.Context
import android.net.Uri
import android.provider.MediaStore
import com.dxkj.myshell.data.db.HostEntity
import com.dxkj.myshell.data.repo.KeyRepository
import com.dxkj.myshell.ui.screens.RemoteEntryUi
import com.dxkj.myshell.crypto.CryptoManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.util.Locale
import net.schmizz.sshj.SSHClient
import net.schmizz.sshj.sftp.FileAttributes
import net.schmizz.sshj.sftp.SFTPClient
import net.schmizz.sshj.transport.TransportException
import net.schmizz.sshj.transport.verification.PromiscuousVerifier
import net.schmizz.sshj.userauth.UserAuthException
import net.schmizz.sshj.userauth.password.PasswordFinder
import net.schmizz.sshj.userauth.password.Resource
import java.io.File
import java.io.FileOutputStream
import java.net.ConnectException
import java.net.SocketException
import java.net.SocketTimeoutException
import java.net.UnknownHostException
import com.dxkj.myshell.ssh.SshCompatConfig
import android.provider.OpenableColumns
import net.schmizz.sshj.sftp.RemoteFile
import net.schmizz.sshj.sftp.SFTPException
import java.io.ByteArrayOutputStream
import java.nio.charset.Charset
import android.util.Log
data class ListResult(val ok: Boolean, val message: String, val entries: List<RemoteEntryUi>)
class SftpClientManager(
    private val keyRepo: KeyRepository,
) {
    private var client: SSHClient? = null
    private var sftp: SFTPClient? = null
    suspend fun connect(host: HostEntity): com.dxkj.myshell.ssh.ConnectResult {
        disconnect()
        val c = SSHClient(SshCompatConfig.create())
        c.addHostKeyVerifier(PromiscuousVerifier())
        c.connectTimeout = 10_000
        c.timeout = 15_000
        return try {
            try {
                c.connect(host.host, host.port)
            } catch (t: Throwable) {
```

```
                return com.dxkj.myshell.ssh.ConnectResult(false, formatFailure("TCP/握手", host, t))
            }
            when (host.authType) {
                "password" -> {
                    try {
                        val pwd = CryptoManager.decryptFromBase64(host.passwordEnc) ?: ""
                        c.authPassword(host.username, pwd)
                    } catch (t: Throwable) {
                        return com.dxkj.myshell.ssh.ConnectResult(false, formatFailure(" 认证", host, t))
                    }
                }
                "key" -> {
                    val keyId = host.privateKeyId ?: return com.dxkj.myshell.ssh.ConnectResult(false, " 未关联密钥
                    val key = keyRepo.getDecryptedById(keyId) ?: return com.dxkj.myshell.ssh.ConnectResult(false, "
                    val finder = object : PasswordFinder {
                        override fun reqPassword(resource: Resource<*>): CharArray = key.passphrase?.toCharArray()
                        override fun shouldRetry(resource: Resource<*>): Boolean = false
                    }
                    val kp = c.loadKeys(key.privateKeyPem, null, finder)
                    try {
                        c.authPublickey(host.username, kp)
                    } catch (t: Throwable) {
                        return com.dxkj.myshell.ssh.ConnectResult(false, formatFailure(" 认证", host, t))
                    }
                }
                else -> return com.dxkj.myshell.ssh.ConnectResult(false, " 未知认证方式: ${host.authType}")
            }
            client = c
            sftp = c.newSFTPClient()
            com.dxkj.myshell.ssh.ConnectResult(true, " 连接成功")
        } catch (t: Throwable) {
            try {
                c.disconnect()
            } catch (_: Throwable) {
            }
            try {
                c.close()
            } catch (_: Throwable) {
            }
            com.dxkj.myshell.ssh.ConnectResult(false, formatFailure(" 未知阶段", host, t))
        }
    }
    private fun formatFailure(stage: String, host: HostEntity, t: Throwable): String {
        val type = t::class.java.simpleName
        val msg = (t.message ?: "").trim()
        val hint = when (t) {
            is UnknownHostException -> "（域名无法解析）"
            is ConnectException -> "（无法建立 TCP 连接：端口不通/被防火墙拦截/地址不对）"
            is SocketTimeoutException -> "（连接超时）"
            is SocketException -> "（底层 Socket 错误：常见于服务器主动断开/Connection reset）"
```

```
            is UserAuthException -> "（认证失败：密码/密钥不对，或服务器禁用该认证方式）"
            is TransportException -> "（传输层错误：常见于算法不兼容/握手被断开）"
            else -> ""
        }
        return buildString {
            append(" 主机 =").append(host.username).append("@").append(host.host).append(":").append(host.port)
            append(", 阶段 =").append(stage)
            append(", 异常 =").append(type)
            if (hint.isNotEmpty()) append(hint)
            if (msg.isNotEmpty()) append(", 详情 =").append(msg)
        }
    }
    suspend fun list(path: String): ListResult = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext ListResult(false, " 未连接", emptyList())
        val p = path.ifBlank { "/" }
        return@withContext try {
            val ls = s.ls(p)
            val entries = ls
                .filter { it.name != "." && it.name != ".." }
                .map {
                    val modeOctal = runCatching {
                        val attrs = it.attributes
                        if (!attrs.has(FileAttributes.Flag.MODE)) return@runCatching ""
                        val perms = attrs.mode.permissionsMask
                        if (perms == 0) "" else String.format(Locale.US, "%o", perms)
                    }.getOrDefault("")
                    RemoteEntryUi(
                        name = it.name,
                        path = if (p.endsWith("/")) p + it.name else "$p/${it.name}",
                        isDir = it.attributes.type.name.contains("DIRECTORY", ignoreCase = true),
                        size = it.attributes.size,
                        modeOctal = modeOctal,
                    )
                }
                .sortedWith(compareByDescending<RemoteEntryUi> { it.isDir }.thenBy { it.name })
            ListResult(true, " 列目录成功: ${entries.size} 项", entries)
        } catch (t: Throwable) {
            ListResult(false, t.message ?: " 列目录失败", emptyList())
        }
    }
    suspend fun homeDir(): Result<String> = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext Result.failure(IllegalStateException(" 未连接"))
        return@withContext runCatching { s.canonicalize(".") }
    }
    suspend fun mkdir(path: String): String = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext " 未连接"
        return@withContext try {
            s.mkdir(path)
            " 创建目录成功: $path"
        } catch (t: Throwable) {
```

```
            " 创建目录失败: ${t.message ?: t::class.java.simpleName}"
        }
    }
    suspend fun rm(remotePath: String): String = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext " 未连接"
        return@withContext try {
            try {
                s.rm(remotePath)
            } catch (_: Throwable) {
                s.rmdir(remotePath)
            }
            " 删除成功: $remotePath"
        } catch (t: Throwable) {
            " 删除失败: ${t.message ?: t::class.java.simpleName}"
        }
    }
    suspend fun rename(oldPath: String, newPath: String): String = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext " 未连接"
        return@withContext try {
            s.rename(oldPath, newPath)
            " 重命名成功"
        } catch (t: Throwable) {
            " 重命名失败: ${t.message ?: t::class.java.simpleName}"
        }
    }
    suspend fun chmod(remotePath: String, modeOctal: String): String = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext " 未连接"
        val trimmed = modeOctal.trim()
        if (trimmed.isEmpty()) return@withContext " 权限不能为空"
        val mode = trimmed.toIntOrNull(8) ?: return@withContext " 无效的八进制权限（如 644、0755）"
        return@withContext try {
            s.chmod(remotePath, mode)
            " 权限已更新: $remotePath → $trimmed"
        } catch (t: Throwable) {
            "chmod 失败: ${t.message ?: t::class.java.simpleName}"
        }
    }
    suspend fun readTextFile(remotePath: String, maxBytes: Int = 512 * 1024): Result<String> = withContext(Dispatch
        val s = sftp ?: return@withContext Result.failure(IllegalStateException(" 未连接"))
        return@withContext try {
            s.open(remotePath).use { rf: RemoteFile ->
                val out = ByteArrayOutputStream()
                val buf = ByteArray(16 * 1024)
                var total = 0
                while (true) {
                    val n = rf.read(total.toLong(), buf, 0, buf.size)
                    if (n <= 0) break
                    total += n
                    if (total > maxBytes) break
                    out.write(buf, 0, n)
```

```
                }
                val bytes = out.toByteArray()
                Result.success(bytes.toString(Charset.forName("UTF-8")))
            }
        } catch (t: Throwable) {
            Result.failure(t)
        }
    }
    suspend fun writeTextFile(remotePath: String, text: String): Result<Unit> = withContext(Dispatchers.IO) {
        val s = sftp ?: return@withContext Result.failure(IllegalStateException(" 未连接"))
        return@withContext try {
            val bytes = text.toByteArray(Charsets.UTF_8)
            s.open(remotePath, setOf(net.schmizz.sshj.sftp.OpenMode.CREAT, net.schmizz.sshj.sftp.OpenMode.TRUNC, ne
                rf.write(0, bytes, 0, bytes.size)
            }
            Result.success(Unit)
        } catch (t: Throwable) {
            Result.failure(t)
        }
    }
    suspend fun downloadToDownloads(
        remotePath: String,
        filename: String,
        context: Context,
        resolver: ContentResolver,
        onProgress: (sent: Long, total: Long) -> Unit = { _, _ -> },
    ): String =
        withContext(Dispatchers.IO) {
            val s = sftp ?: return@withContext " 未连接"
            val safeName = filename.ifBlank { "download.bin" }
            val total = runCatching { s.size(remotePath) }.getOrElse { -1L }
            fun doDownloadToOutput(openOut: () -> java.io.OutputStream): String {
                val dest = OutputStreamDestFile(
                    name = safeName,
                    length = total,
                    open = { _ -> openOut() },
                    onProgress = { bytes -> onProgress(bytes, total) },
                )
                s.get(remotePath, dest)
                return " 下载完成: $safeName"
            }
            val mediaStoreUri: Uri? = runCatching {
                val values = ContentValues().apply {
                    put(MediaStore.Downloads.DISPLAY_NAME, safeName)
                    put(MediaStore.Downloads.MIME_TYPE, "application/octet-stream")
                    put(MediaStore.Downloads.RELATIVE_PATH, "Download/MyShell")
                }
                resolver.insert(MediaStore.Downloads.EXTERNAL_CONTENT_URI, values)
            }.getOrNull()
            if (mediaStoreUri != null) {
```

```
                val r = runCatching {
                    doDownloadToOutput {
                        resolver.openOutputStream(mediaStoreUri) ?: throw IllegalStateException(" 无法写入下载文件
                    }
                }
                if (r.isSuccess) return@withContext r.getOrThrow()
                try {
                    resolver.delete(mediaStoreUri, null, null)
                } catch (_: Throwable) {
                }
                Log.w("MyShell-SFTP", "download MediaStore failed: path=$remotePath name=$safeName", r.exceptionOrN
            }
            return@withContext runCatching {
                val dir = File(context.filesDir, "downloads/MyShell").apply { mkdirs() }
                val outFile = File(dir, safeName)
                "${doDownloadToOutput { FileOutputStream(outFile) }}（已保存到应用内: ${outFile.name}）"
            }.getOrElse { t ->
                Log.e("MyShell-SFTP", "download fallback failed: path=$remotePath name=$safeName", t)
                " 下载失败: ${t::class.java.simpleName}: ${t.message ?: "unknown"}"
            }
        }
    suspend fun uploadFromUri(
        remoteDir: String,
        uri: Uri,
        resolver: ContentResolver,
        onProgress: (sent: Long, total: Long) -> Unit = { _, _ -> },
    ): String =
        withContext(Dispatchers.IO) {
            val s = sftp ?: return@withContext " 未连接"
            val dir = remoteDir.ifBlank { "/" }
            val name = queryDisplayName(resolver, uri) ?: "upload.bin"
            fun buildRemotePath(d: String): String = if (d.endsWith("/")) d + name else "$d/$name"
            val dirCanon = runCatching { s.canonicalize(dir) }.getOrNull() ?: dir
            val remotePath = buildRemotePath(dirCanon)
            return@withContext try {
                val total = queryLength(resolver, uri)
                val src = InputStreamSourceFile(
                    name = name,
                    length = total,
                    open = { resolver.openInputStream(uri) ?: throw IllegalStateException(" 无法读取要上传的文件")
                    onProgress = { bytes -> onProgress(bytes, total) },
                )
                s.put(src, remotePath)
                " 上传完成: $remotePath"
            } catch (t: Throwable) {
                val msg = (t.message ?: "").trim().ifBlank { t::class.java.simpleName }
                val code = (t as? SFTPException)?.statusCode
                val home = runCatching { s.canonicalize(".") }.getOrNull()
                Log.w(
                    "MyShell-SFTP",
```

```
                    "upload failed: dir=$dir canon=$dirCanon target=$remotePath home=$home type=${t::class.java.sim
                    t,
                )
                buildString {
                    append(" 上传失败: ").append(msg)
                    if (code != null) append(" (SFTP code=").append(code).append(") ")
                    append(" (目标 =").append(remotePath).append(") ")
                    if (!home.isNullOrBlank()) append(" (home=").append(home).append(") ")
                    append("。如果你确认目录可写，通常是路径解析/服务端策略导致；请尝试先刷新目录后再上传。")
                }
            }
        }
    private fun queryDisplayName(resolver: ContentResolver, uri: Uri): String? {
        return try {
            resolver.query(uri, arrayOf(android.provider.OpenableColumns.DISPLAY_NAME), null, null, null)?.use { c
                if (c.moveToFirst()) {
                    val idx = c.getColumnIndex(android.provider.OpenableColumns.DISPLAY_NAME)
                    if (idx >= 0) c.getString(idx) else null
                } else null
            }
        } catch (_: Throwable) {
            null
        }
    }
    private fun queryLength(resolver: ContentResolver, uri: Uri): Long {
        return try {
            resolver.query(uri, arrayOf(OpenableColumns.SIZE), null, null, null)?.use { c ->
                if (c.moveToFirst()) {
                    val idx = c.getColumnIndex(OpenableColumns.SIZE)
                    if (idx >= 0) c.getLong(idx) else -1L
                } else -1L
            } ?: -1L
        } catch (_: Throwable) {
            -1L
        }
    }
    suspend fun disconnect() {
        try {
            sftp?.close()
        } catch (_: Throwable) {
        } finally {
            sftp = null
        }
        try {
            client?.disconnect()
        } catch (_: Throwable) {
        }
        try {
            client?.close()
        } catch (_: Throwable) {
```

```
        } finally {
            client = null
        }
    }
}
package com.dxkj.myshell.ssh
import kotlin.math.roundToInt
data class LinuxHostMetricsSnapshot(
    val load1: Float?,
    val load5: Float?,
    val load15: Float?,
    val memTotalKb: Long?,
    val memAvailKb: Long?,
    val memUsedPct: Float?,
    val swapTotalKb: Long?,
    val swapFreeKb: Long?,
    val swapUsedPct: Float?,
    val rootDiskUsePct: Float?,
    val processLines: List<String>,
)
object RemoteLinuxMetrics {
    val remoteCollectCommand: String =
        "/bin/sh -c 'export LANG=C LC_ALL=C; " +
            "echo ---LOAD---; cat /proc/loadavg 2>/dev/null || echo NA; " +
            "echo ---MEM---; " +
            "grep ^MemTotal: /proc/meminfo 2>/dev/null; " +
            "grep ^MemAvailable: /proc/meminfo 2>/dev/null; " +
            "grep ^MemFree: /proc/meminfo 2>/dev/null; " +
            "grep ^SwapTotal: /proc/meminfo 2>/dev/null; " +
            "grep ^SwapFree: /proc/meminfo 2>/dev/null; " +
            "echo ---DISK---; df -Pk / 2>/dev/null | tail -n +2 | head -n 1; " +
            "echo ---PS---; ps aux 2>/dev/null | head -n 7'"
    suspend fun collect(ssh: SshSessionManager): Result<LinuxHostMetricsSnapshot> =
        ssh.execRemoteCapture(remoteCollectCommand, timeoutSec = 18L).map { parseCollectorOutput(it) }
    private fun extractSection(text: String, start: String, end: String?): String {
        val i = text.indexOf(start)
        if (i < 0) return ""
        val from = i + start.length
        val j = if (end != null) text.indexOf(end, from) else -1
        return if (j >= 0) text.substring(from, j).trim() else text.substring(from).trim()
    }
    private fun parseKbFromMeminfoLine(line: String): Long? {
        val m = Regex("(\\d+)\\s*kB").find(line) ?: return null
        return m.groupValues[1].toLongOrNull()
    }
    fun parseCollectorOutput(output: String): LinuxHostMetricsSnapshot {
        val loadBlock = extractSection(output, "---LOAD---", "---MEM---").lines().firstOrNull { it.isNotBlank() }?.
        val (l1, l5, l15) = parseLoadavg(loadBlock)
        val memBlock = extractSection(output, "---MEM---", "---DISK---")
        var memTotal: Long? = null
```

```
        var memAvail: Long? = null
        var memFree: Long? = null
        var swapTotal: Long? = null
        var swapFree: Long? = null
        for (line in memBlock.lines()) {
            val t = line.trim()
            when {
                t.startsWith("MemTotal:") -> memTotal = parseKbFromMeminfoLine(t)
                t.startsWith("MemAvailable:") -> memAvail = parseKbFromMeminfoLine(t)
                t.startsWith("MemFree:") -> memFree = parseKbFromMeminfoLine(t)
                t.startsWith("SwapTotal:") -> swapTotal = parseKbFromMeminfoLine(t)
                t.startsWith("SwapFree:") -> swapFree = parseKbFromMeminfoLine(t)
            }
        }
        val avail = memAvail ?: memFree
        val memUsedPct = if (memTotal != null && memTotal > 0L && avail != null) {
            (((memTotal - avail).coerceAtLeast(0L)).toFloat() / memTotal.toFloat() * 100f).coerceIn(0f, 100f)
        } else {
            null
        }
        val swapUsedPct = if (swapTotal != null && swapTotal > 0L && swapFree != null) {
            (((swapTotal - swapFree).coerceAtLeast(0L)).toFloat() / swapTotal.toFloat() * 100f).coerceIn(0f, 100f)
        } else {
            null
        }
        val diskLine = extractSection(output, "---DISK---", "---PS---").lines().firstOrNull { it.isNotBlank() }?.tr
        val diskPct = parseDfCapacityPct(diskLine)
        val psBlock = extractSection(output, "---PS---", null)
        val procLines = psBlock.lines()
            .map { it.trim() }
            .filter { it.isNotEmpty() }
            .filterNot { it.startsWith("USER ") || it.startsWith("PID ") }
        return LinuxHostMetricsSnapshot(
            load1 = l1,
            load5 = l5,
            load15 = l15,
            memTotalKb = memTotal,
            memAvailKb = avail,
            memUsedPct = memUsedPct,
            swapTotalKb = swapTotal,
            swapFreeKb = swapFree,
            swapUsedPct = swapUsedPct,
            rootDiskUsePct = diskPct,
            processLines = procLines.take(6),
        )
    }
    private fun parseLoadavg(line: String): Triple<Float?, Float?, Float?> {
        if (line.isBlank() || line == "NA") return Triple(null, null, null)
        val parts = line.split(Regex("\\s+")).filter { it.isNotBlank() }
        if (parts.size < 3) return Triple(null, null, null)
```

```
        val a = parts[0].toFloatOrNull()
        val b = parts[1].toFloatOrNull()
        val c = parts[2].toFloatOrNull()
        return Triple(a, b, c)
    }
    private fun parseDfCapacityPct(line: String): Float? {
        if (line.isBlank() || line == "NODISK") return null
        val tok = line.split(Regex("\\s+")).filter { it.isNotBlank() }
        val pctTok = tok.firstOrNull { it.endsWith("%") } ?: return null
        return pctTok.trimEnd('%').toFloatOrNull()
    }
    fun formatPct(p: Float?): String =
        if (p == null) "—" else "${p.roundToInt()}%"
    fun formatLoad(a: Float?, b: Float?, c: Float?): String =
        if (a == null && b == null && c == null) "—" else listOfNotNull(a, b, c).joinToString(" / ") { String.form
}
package com.dxkj.myshell.ssh
import java.util.regex.Pattern
object RemotePortDiscovery {
    private val ssListenLine = Pattern.compile(
        """^(?:(?:tcp6?|sctp)\s+)?LISTEN\s+\d+\s+\d+\s+((?:\[[^\]]+\]|[^\s:]+)):(\d+)\s+""",
    )
    private val terminalPatterns = listOf(
        Pattern.compile(
            """(?i)https?://(?:localhost|127\.0\.0\.1|\[::1\])(?::(\d{2,5}))\b""",
        ),
        Pattern.compile(
            """(?i)(?:^|[^\w.])(?:localhost|127\.0\.0\.1|\[::1\])(?::(\d{2,5}))\b""",
        ),
        Pattern.compile("""(?i)(?:0\.0\.0\.0|\*|::|\[::\])(?::(\d{2,5}))\b"""),
        Pattern.compile("""(?i)Local:\s*https?://[^\s]*?:(\d{2,5})\b"""),
        Pattern.compile("""(?i)\bport\s+(\d{2,5})\s+(?:is\s+)?(?:open|in\s+use|listening)"""),
    )
    fun parseSsListenTcp(output: String): List<Pair<String, Int>> {
        val out = LinkedHashSet<Pair<String, Int>>()
        for (line in output.lineSequence()) {
            val m = ssListenLine.matcher(line)
            if (!m.find()) continue
            val addrRaw = m.group(1) ?: continue
            val port = m.group(2)?.toIntOrNull() ?: continue
            if (port !in 1..65535) continue
            val addr = normalizeListenAddr(addrRaw)
            if (addr.isNotEmpty()) out.add(addr to port)
        }
        return out.toList()
    }
    private val netstatListen = Pattern.compile(
        """^(?:tcp|tcp6)\s+\S+\s+\S+\s+(\S+):(\d+)\s+\S+\s+LISTEN\s*$""",
    )
    fun parseNetstatListenTcp(output: String): List<Pair<String, Int>> {
```

```
        val out = LinkedHashSet<Pair<String, Int>>()
        for (line in output.lineSequence()) {
            val t = line.trim()
            val m = netstatListen.matcher(t)
            if (!m.find()) continue
            val addrRaw = m.group(1) ?: continue
            val port = m.group(2)?.toIntOrNull() ?: continue
            if (port !in 1..65535) continue
            val addr = normalizeListenAddr(addrRaw)
            if (addr.isNotEmpty()) out.add(addr to port)
        }
        return out.toList()
    }
    fun allowAutoForwardFromRemoteScan(port: Int): Boolean {
        if (port !in 1..65535) return false
        return port !in setOf(22, 53, 111, 631, 25, 110, 143, 993, 995)
    }
    fun extractFromTerminalChunk(chunk: String): List<Pair<String, Int>> {
        if (chunk.length < 4) return emptyList()
        val found = LinkedHashSet<Pair<String, Int>>()
        for (p in terminalPatterns) {
            val m = p.matcher(chunk)
            while (m.find()) {
                val g1 = m.group(1) ?: continue
                val port = g1.toIntOrNull() ?: continue
                if (port !in 1..65535) continue
                found.add("127.0.0.1" to port)
            }
        }
        return found.toList()
    }
    private fun normalizeListenAddr(addrPart: String): String {
        val s = addrPart.trim()
        if (s == "*" || s == "0.0.0.0" || s == "::" || s == "[::]" || s == "[::1]" || s == "::1") {
            return "127.0.0.1"
        }
        if (s.startsWith("[::ffff:") && s.contains("127.0.0.1")) return "127.0.0.1"
        if (s.matches(Regex("^[0-9]{1,3}(\\.[0-9]{1,3}){3}$"))) return s
        if (s.startsWith("[")) return s
        return s
    }
}
package com.dxkj.myshell.ssh
import net.schmizz.sshj.DefaultConfig
import net.schmizz.sshj.transport.cipher.AES128CTR
import net.schmizz.sshj.transport.cipher.AES256CTR
import net.schmizz.sshj.transport.cipher.TripleDESCBC
import net.schmizz.sshj.transport.kex.Curve25519SHA256
import net.schmizz.sshj.transport.kex.DHG1
import net.schmizz.sshj.transport.kex.DHG14
```

```
import net.schmizz.sshj.transport.kex.DHGexSHA1
import net.schmizz.sshj.transport.kex.DHGexSHA256
import net.schmizz.sshj.transport.mac.HMACSHA1
import net.schmizz.sshj.transport.mac.HMACSHA2256
object SshCompatConfig {
    fun create(): DefaultConfig {
        val config = DefaultConfig()
        config.setKeyExchangeFactories(
            listOf(
                Curve25519SHA256.Factory(),
                DHGexSHA256.Factory(),
                DHG14.Factory(),
                DHGexSHA1.Factory(),
                DHG1.Factory(),
            ),
        )
        config.setCipherFactories(
            listOf(
                AES256CTR.Factory(),
                AES128CTR.Factory(),
                TripleDESCBC.Factory(),
            ),
        )
        config.setMACFactories(
            listOf(
                HMACSHA2256.Factory(),
                HMACSHA1.Factory(),
            ),
        )
        config.prioritizeSshRsaKeyAlgorithm()
        return config
    }
}
package com.dxkj.myshell.ssh
import com.dxkj.myshell.data.db.HostEntity
import com.dxkj.myshell.data.repo.KeyRepository
import com.dxkj.myshell.crypto.CryptoManager
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.cancel
import kotlinx.coroutines.cancelChildren
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import net.schmizz.sshj.SSHClient
import net.schmizz.sshj.connection.channel.direct.LocalPortForwarder
```

```
import net.schmizz.sshj.connection.channel.direct.Parameters
import net.schmizz.sshj.connection.channel.direct.Session
import net.schmizz.sshj.transport.TransportException
import net.schmizz.sshj.transport.verification.PromiscuousVerifier
import net.schmizz.sshj.userauth.password.Resource
import net.schmizz.sshj.userauth.password.PasswordFinder
import net.schmizz.sshj.userauth.UserAuthException
import java.net.ConnectException
import java.net.InetAddress
import java.net.InetSocketAddress
import java.net.ServerSocket
import java.net.SocketException
import java.net.SocketTimeoutException
import java.net.UnknownHostException
import java.io.InputStream
import java.io.OutputStream
import java.util.LinkedHashMap
import java.util.concurrent.ConcurrentHashMap
import java.util.concurrent.TimeUnit
import java.util.concurrent.atomic.AtomicLong
data class ConnectResult(val ok: Boolean, val message: String)
data class PortForwardItem(
    val id: Long,
    val localHost: String,
    val localPort: Int,
    val remoteHost: String,
    val remotePort: Int,
)
data class DiscoveredListen(
    val remoteHost: String,
    val remotePort: Int,
    val source: String,
)
private data class ForwardRuntime(
    val item: PortForwardItem,
    val forwarder: LocalPortForwarder,
    val serverSocket: ServerSocket,
)
private data class ForwardRestoreRule(
    val remoteHost: String,
    val remotePort: Int,
    val preferredLocalPort: Int,
)
class SshSessionManager(
    private val keyRepo: KeyRepository,
    private val onPortForwardUiChanged: () -> Unit = {},
) {
    @Volatile private var lastPtyCols: Int = 80
    @Volatile private var lastPtyRows: Int = 24
    private var client: SSHClient? = null
```

```
    private var session: Session? = null
    private var shell: Session.Shell? = null
    private var writer: java.io.OutputStream? = null
    private var reader: java.io.InputStream? = null
    private var readerJob: Job? = null
    private var scope: CoroutineScope? = null
    private val forwardSupervisor = SupervisorJob()
    private val forwardScope = CoroutineScope(forwardSupervisor + Dispatchers.IO)
    private val forwardIdGen = AtomicLong(1)
    private val activeForwards = ConcurrentHashMap<Long, ForwardRuntime>()
    private val restoreRules = ConcurrentHashMap<String, ForwardRestoreRule>()
    private val _portForwards = MutableStateFlow<List<PortForwardItem>>(emptyList())
    val portForwards: StateFlow<List<PortForwardItem>> = _portForwards.asStateFlow()
    private val discoveredMap = LinkedHashMap<String, DiscoveredListen>()
    private val _discoveredList = MutableStateFlow<List<DiscoveredListen>>(emptyList())
    val discoveredList: StateFlow<List<DiscoveredListen>> = _discoveredList.asStateFlow()
    private val terminalSniffBuf = StringBuilder(16_384)
    private val terminalSeenKeys = mutableSetOf<String>()
    private val ignoredRemoteKeys = java.util.Collections.newSetFromMap(ConcurrentHashMap<String, Boolean>())
    private val _ignoredRemotePorts = MutableStateFlow<Set<String>>(emptySet())
    val ignoredRemotePorts: StateFlow<Set<String>> = _ignoredRemotePorts.asStateFlow()
    private fun remotePortKey(host: String, port: Int) = "${host.trim().lowercase()}:$port"
    private fun refreshIgnoredFlow() {
        _ignoredRemotePorts.value = ignoredRemoteKeys.toSet()
        notifyPortForwardUiChanged()
    }
    fun isRemotePortIgnored(remoteHost: String, remotePort: Int): Boolean =
        ignoredRemoteKeys.contains(remotePortKey(remoteHost, remotePort))
    @Synchronized
    fun removeDiscoveredFromListOnly(host: String, port: Int) {
        val key = remotePortKey(host, port)
        if (discoveredMap.remove(key) != null) {
            refreshDiscoveredFlow()
        }
    }
    fun clearAllDismissedRemotePorts() {
        ignoredRemoteKeys.clear()
        refreshIgnoredFlow()
    }
    @Synchronized
    fun clearIgnoredRemote(remoteHost: String, remotePort: Int) {
        val key = remotePortKey(remoteHost, remotePort)
        if (ignoredRemoteKeys.remove(key)) {
            refreshIgnoredFlow()
        }
    }
    private fun refreshDiscoveredFlow() {
        _discoveredList.value = discoveredMap.values.reversed()
        notifyPortForwardUiChanged()
    }
```

```
    @Synchronized
    fun mergeDiscoveredPort(host: String, port: Int, source: String) {
        val h = host.trim().ifEmpty { return }
        if (port !in 1..65535) return
        if (port == SSH_CONTROL_PORT) return
        val key = "${h.lowercase()}:$port"
        if (ignoredRemoteKeys.contains(key)) return
        discoveredMap.remove(key)
        discoveredMap[key] = DiscoveredListen(h, port, source)
        while (discoveredMap.size > 80) {
            val first = discoveredMap.keys.first()
            discoveredMap.remove(first)
        }
        refreshDiscoveredFlow()
    }
    @Synchronized
    fun feedTerminalSniffForPorts(bytes: ByteArray, off: Int, len: Int): List<Pair<String, Int>> {
        if (len <= 0) return emptyList()
        val chunk = String(bytes, off, len, Charsets.UTF_8)
        terminalSniffBuf.append(chunk)
        if (terminalSniffBuf.length > 20_000) {
            terminalSniffBuf.delete(0, terminalSniffBuf.length - 10_000)
        }
        val text = terminalSniffBuf.toString()
        val extracted = RemotePortDiscovery.extractFromTerminalChunk(text)
        val news = mutableListOf<Pair<String, Int>>()
        for ((h, p) in extracted) {
            if (p == SSH_CONTROL_PORT) continue
            val k = "${h.lowercase()}:$p"
            if (ignoredRemoteKeys.contains(k)) continue
            mergeDiscoveredPort(h, p, " 终端")
            if (terminalSeenKeys.add(k)) {
                news.add(h to p)
            }
        }
        return news
    }
    @Synchronized
    private fun clearPortDiscoveryState() {
        discoveredMap.clear()
        _discoveredList.value = emptyList()
        terminalSeenKeys.clear()
        terminalSniffBuf.clear()
        ignoredRemoteKeys.clear()
        _ignoredRemotePorts.value = emptySet()
        notifyPortForwardUiChanged()
    }
    fun isForwardingRemote(remoteHost: String, remotePort: Int): Boolean {
        val h = remoteHost.trim().lowercase()
        return activeForwards.values.any {
```

```
            it.item.remotePort == remotePort && it.item.remoteHost.trim().lowercase() == h
        }
    }
    suspend fun startSamePortForwardIfAbsent(
        remoteHost: String,
        remotePort: Int,
        explicitUserAction: Boolean = true,
    ): Result<PortForwardItem> {
        if (remotePort == SSH_CONTROL_PORT) {
            return Result.failure(IllegalArgumentException("22 为 SSH 端口，无需转发"))
        }
        val key = remotePortKey(remoteHost, remotePort)
        if (!explicitUserAction && ignoredRemoteKeys.contains(key)) {
            return Result.failure(IllegalStateException(" 已忽略"))
        }
        if (explicitUserAction) {
            ignoredRemoteKeys.remove(key)
            refreshIgnoredFlow()
        }
        if (isForwardingRemote(remoteHost, remotePort)) {
            return Result.failure(IllegalStateException(" 已在转发"))
        }
        var r = startLocalPortForward(remotePort, remoteHost, remotePort)
        if (r.isFailure) {
            r = startLocalPortForward(0, remoteHost, remotePort)
        }
        return r
    }
    suspend fun scanRemoteListenersAndMerge(autoSamePortForward: Boolean): Result<Int> =
        withContext(Dispatchers.IO) {
            val c = client ?: return@withContext Result.failure(IllegalStateException(" 未连接"))
            val r = execCaptureUnscoped(
                c,
                "bash -lc 'export LANG=C LC_ALL=C; ss -Hltn 2>/dev/null || netstat -lnt 2>/dev/null || true'",
                timeoutSec = 20L,
            )
            if (r.isFailure) {
                val e = r.exceptionOrNull() ?: Exception(" 扫描失败")
                return@withContext Result.failure(e)
            }
            val text = r.getOrNull() ?: ""
            var list = RemotePortDiscovery.parseSsListenTcp(text)
            if (list.isEmpty()) {
                list = RemotePortDiscovery.parseNetstatListenTcp(text)
            }
            for ((h, p) in list) {
                mergeDiscoveredPort(h, p, " 远端扫描")
            }
            if (autoSamePortForward) {
                for ((h, p) in list) {
```

```
                if (!RemotePortDiscovery.allowAutoForwardFromRemoteScan(p)) continue
                startSamePortForwardIfAbsent(h, p, explicitUserAction = false)
            }
        }
        Result.success(list.size)
    }
suspend fun execRemoteCapture(command: String, timeoutSec: Long = 12L): Result<String> =
    withContext(Dispatchers.IO) {
        val c = client ?: return@withContext Result.failure(IllegalStateException(" 未连接"))
        execCaptureUnscoped(c, command, timeoutSec)
    }
private suspend fun execCaptureUnscoped(c: SSHClient, command: String, timeoutSec: Long = 12L): Result<String>
    withContext(Dispatchers.IO) {
        val sess = c.startSession()
        try {
            val cmd = sess.exec(command)
            val t = timeoutSec.coerceIn(3L, 120L)
            cmd.join(t, TimeUnit.SECONDS)
            val out = cmd.inputStream.use { ins -> ins.readBytes().toString(Charsets.UTF_8) }
            val err = cmd.errorStream.use { es -> es.readBytes().toString(Charsets.UTF_8) }
            val merged = listOf(out, err).filter { it.isNotBlank() }.joinToString("\n")
            Result.success(merged)
        } catch (t: Throwable) {
            Result.failure(t)
        } finally {
            try {
                sess.close()
            } catch (_: Throwable) {
            }
        }
    }
private fun refreshPortForwardFlow() {
    _portForwards.value = activeForwards.values.map { it.item }.sortedWith(compareBy({ it.localPort }, { it.id
    notifyPortForwardUiChanged()
}
private fun notifyPortForwardUiChanged() {
    try {
        onPortForwardUiChanged()
    } catch (_: Throwable) {
    }
}
fun activeLocalPortForwardCount(): Int = activeForwards.size
private fun rememberRestoreRule(remoteHost: String, remotePort: Int, boundLocalPort: Int) {
    val h = remoteHost.trim()
    if (h.isEmpty() || remotePort !in 1..65535 || remotePort == SSH_CONTROL_PORT || boundLocalPort !in 1..65535
    val key = remotePortKey(h, remotePort)
    if (ignoredRemoteKeys.contains(key)) return
    restoreRules[key] = ForwardRestoreRule(h, remotePort, boundLocalPort)
}
private fun forgetRestoreRule(remoteHost: String, remotePort: Int) {
```

```
                    ),
                )
            }
        },
        confirmButton = {
            TextButton(onClick = { onConfirm(displaySize) }) {
                Text(" 确定", fontWeight = FontWeight.Medium)
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text(" 取消")
            }
        },
    )
}
package com.dxkj.myshell.ui.screens
import android.app.Activity
import android.app.Application
import android.content.Context
import android.view.WindowManager
import android.view.View
import android.view.ViewConfiguration
import android.graphics.Typeface
import androidx.activity.compose.BackHandler
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.focusable
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.systemBarsPadding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.outlined.ArrowBack
import androidx.compose.material.icons.outlined.Add
import androidx.compose.material.icons.outlined.PowerSettingsNew
import androidx.compose.material.icons.outlined.Refresh
import androidx.compose.material.icons.outlined.ContentPaste
import androidx.compose.material.icons.outlined.Palette
import androidx.compose.material.icons.outlined.Contrast
```

```
import androidx.compose.material.icons.outlined.Remove
import androidx.compose.material.icons.outlined.ContentCopy
import androidx.compose.material.icons.outlined.Save
import androidx.compose.material.icons.outlined.MoreVert
import androidx.compose.material.icons.outlined.KeyboardArrowDown
import androidx.compose.material.icons.outlined.Keyboard
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.FilledTonalIconButton
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.key
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.focusRequester
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.platform.LocalClipboardManager
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.AnnotatedString
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.ui.focus.FocusRequester
import androidx.compose.ui.unit.sp
import androidx.core.content.res.ResourcesCompat
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsCompat
import androidx.core.view.WindowInsetsControllerCompat
import com.dxkj.myshell.R
import com.dxkj.myshell.data.db.DbProvider
import com.dxkj.myshell.data.prefs.AppPreferences
import com.dxkj.myshell.data.repo.HostRepository
import com.dxkj.myshell.terminal.TerminalSessionPool
import com.dxkj.myshell.ui.terminal.HavenKeyboardToolbar
import com.dxkj.myshell.ui.terminal.SimpleModifierManager
import com.dxkj.myshell.ui.theme.Dimens
import org.connectbot.terminal.Terminal
import org.connectbot.terminal.SelectionController
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.delay
import android.util.Log
```

```
private fun readIntFieldNoThrow(obj: Any, fieldName: String): Int? {
    return try {
        var cls: Class<*>? = obj::class.java
        while (cls != null) {
            try {
                val f = cls.getDeclaredField(fieldName)
                f.isAccessible = true
                return (f.get(obj) as? Int)
            } catch (_: NoSuchFieldException) {
                cls = cls.superclass
            }
        }
        null
    } catch (_: Throwable) {
        null
    }
}
private fun getLineHeightPxNoThrow(view: View): Int {
    val h = readIntFieldNoThrow(view, "mCharacterHeight")
        ?: readIntFieldNoThrow(view, "mCharHeight")
        ?: readIntFieldNoThrow(view, "mFontHeight")
    return (h ?: 0).coerceAtLeast(0)
}
@Composable
fun TerminalHubScreen(
    initialHostId: Long?,
    onExit: () -> Unit,
    immersive: Boolean = true,
    showBack: Boolean = true,
    showTopOverlay: Boolean = true,
    compactTopOverlay: Boolean = false,
    onToggleTopOverlay: (() -> Unit)? = null,
    embeddedInSessionsPane: Boolean = false,
    embeddedBottomToolbarExpanded: Boolean = false,
) {
    val context = LocalContext.current
    val activity = context as? Activity
    val clipboard = LocalClipboardManager.current
    TerminalSessionPool.init(context.applicationContext as Application)
    val termScheme by AppPreferences.terminalColorScheme.collectAsState()
    val terminalFontSp by AppPreferences.terminalFontSize.collectAsState()
    val termBg = AppPreferences.argbLongToColor(termScheme.background)
    val termFg = AppPreferences.argbLongToColor(termScheme.foreground)
    val hackTypeface = remember(context) {
        try {
            ResourcesCompat.getFont(context, R.font.hack_regular) ?: Typeface.MONOSPACE
        } catch (_: Throwable) {
            Typeface.MONOSPACE
        }
    }
```

```
    val sessions by TerminalSessionPool.sessions.collectAsState()
    val activeId by TerminalSessionPool.activeSessionId.collectAsState()
    val active = sessions.firstOrNull { it.sessionId == activeId } ?: sessions.lastOrNull()
    val prefs = remember(context) { context.getSharedPreferences("terminal_prefs", Context.MODE_PRIVATE) }
    val copyOnSelect by AppPreferences.terminalCopyOnSelect.collectAsState()
    var keyBarVisibleStandalone by remember { mutableStateOf(true) }
    var showHostPicker by remember { mutableStateOf(false) }
    var showMoreMenu by remember { mutableStateOf(false) }
    var showCopyHint by remember { mutableStateOf(false) }
    var copyHintText by remember { mutableStateOf(" 已复制到剪贴板") }
    fun showHint(text: String) {
        copyHintText = text
        showCopyHint = true
    }
    val keyBarVisible = if (embeddedInSessionsPane) {
        embeddedBottomToolbarExpanded
    } else {
        keyBarVisibleStandalone
    }
    val toolbarVisible = keyBarVisible
    val effectiveToolbarVisible = toolbarVisible
    val bottomBarHeight: Dp = if (effectiveToolbarVisible) Dimens.TerminalKeyBarHeight else 0.dp
    LaunchedEffect(initialHostId) {
        val hid = initialHostId?.takeIf { it > 0 } ?: return@LaunchedEffect
        TerminalSessionPool.ensureSession(hid)
    }
    LaunchedEffect(Unit) {
        if (initialHostId == null && sessions.isEmpty()) {
            val ids = TerminalSessionPool.loadOpenHostIds()
            ids.forEach { TerminalSessionPool.ensureSession(it) }
        }
    }
    LaunchedEffect(active?.hostId, embeddedInSessionsPane) {
        val hid = active?.hostId ?: return@LaunchedEffect
        if (!embeddedInSessionsPane) {
            keyBarVisibleStandalone = prefs.getBoolean("keyBar_$hid", true)
        }
    }
    LaunchedEffect(keyBarVisibleStandalone, active?.hostId, embeddedInSessionsPane) {
        val hid = active?.hostId ?: return@LaunchedEffect
        if (!embeddedInSessionsPane) {
            prefs.edit().putBoolean("keyBar_$hid", keyBarVisibleStandalone).apply()
        }
    }
    LaunchedEffect(active?.sessionId, termScheme.name, termFg, termBg) {
        val emu = active?.emulator ?: return@LaunchedEffect
        emu.setDefaultColors(termFg.toArgb(), termBg.toArgb())
    }
    if (showBack) {
        BackHandler { onExit() }
```

```
    }
    if (immersive) {
        DisposableEffect(Unit) {
            if (activity != null) {
                activity.window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
                WindowCompat.setDecorFitsSystemWindows(activity.window, false)
                val controller = WindowInsetsControllerCompat(activity.window, activity.window.decorView)
                controller.systemBarsBehavior =
                    WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
                controller.hide(WindowInsetsCompat.Type.systemBars())
            }
            onDispose {
                if (activity != null) {
                    val controller = WindowInsetsControllerCompat(activity.window, activity.window.decorView)
                    controller.show(WindowInsetsCompat.Type.systemBars())
                    WindowCompat.setDecorFitsSystemWindows(activity.window, true)
                    activity.window.clearFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
                }
            }
        }
    }
    Column(modifier = Modifier.fillMaxSize().background(termBg)) {
        val emulator = active?.emulator
        val focusRequester = remember(active?.sessionId) { FocusRequester() }
        val modifierManager = remember(active?.sessionId) { SimpleModifierManager() }
        Box(modifier = Modifier.weight(1f).fillMaxWidth()) {
            if (emulator != null) {
                key(active!!.sessionId, termScheme.name, terminalFontSp) {
                    var selectionController by remember { mutableStateOf<SelectionController?>(null) }
                    var showIme by remember(active!!.sessionId) { mutableStateOf(false) }
                    LaunchedEffect(active!!.sessionId) {
                        showIme = false
                        delay(200)
                        showIme = true
                    }
                    Terminal(
                        terminalEmulator = emulator,
                        modifier = Modifier
                            .fillMaxSize()
                            .focusable(),
                        typeface = hackTypeface,
                        initialFontSize = terminalFontSp.sp,
                        backgroundColor = termBg,
                        foregroundColor = termFg,
                        keyboardEnabled = true,
                        showSoftKeyboard = showIme,
                        desktopPointerMode = true,
                        copyOnSelect = copyOnSelect,
                        focusRequester = focusRequester,
                        modifierManager = modifierManager,
```

```
                    onSelectionControllerAvailable = { selectionController = it },
                    onPasteRequest = {
                        val t = clipboard.getText()?.text?.toString().orEmpty()
                        if (t.isNotBlank()) {
                            val b = t.toByteArray()
                            TerminalSessionPool.sendBytes(active.sessionId, b)
                        } else {
                            showHint(" 剪贴板为空")
                        }
                    },
                )
            }
        }
        if (active != null && !active.status.isNullOrBlank() && (active.connecting || !active.connected)) {
            Text(
                text = active.status ?: "",
                color = MaterialTheme.colorScheme.inverseOnSurface,
                modifier = Modifier
                    .align(Alignment.BottomCenter)
                    .then(if (immersive) Modifier.systemBarsPadding() else Modifier)
                    .padding(bottom = bottomBarHeight + Dimens.OverlayPaddingH)
                    .background(
                        MaterialTheme.colorScheme.inverseSurface.copy(alpha = 0.86f),
                        RoundedCornerShape(Dimens.OverlayCornerSm),
                    )
                    .padding(horizontal = Dimens.OverlayPaddingH, vertical = Dimens.OverlayPaddingV),
            )
        }
        if (showCopyHint) {
            Text(
                text = copyHintText,
                color = MaterialTheme.colorScheme.inverseOnSurface,
                modifier = Modifier
                    .align(Alignment.BottomCenter)
                    .then(if (immersive) Modifier.systemBarsPadding() else Modifier)
                    .padding(bottom = bottomBarHeight + Dimens.OverlayPaddingH)
                    .background(
                        MaterialTheme.colorScheme.inverseSurface.copy(alpha = 0.86f),
                        RoundedCornerShape(Dimens.OverlayCornerSm),
                    )
                    .padding(horizontal = Dimens.OverlayPaddingH, vertical = Dimens.OverlayPaddingV),
            )
            LaunchedEffect(Unit) {
                kotlinx.coroutines.delay(1200)
                showCopyHint = false
            }
        }
    }
    if (emulator != null && effectiveToolbarVisible) {
        HavenKeyboardToolbar(
```

```
            focusRequester = focusRequester,
            onSendBytes = { bytes ->
                TerminalSessionPool.sendBytes(active.sessionId, bytes)
            },
            onDispatchKey = { _, key ->
                try {
                    emulator.dispatchKey(0, key)
                } catch (_: Throwable) {
                }
            },
            modifier = Modifier
                    .fillMaxWidth(),
            modifierManager = modifierManager,
        )
    } else {
        Spacer(modifier = Modifier.height(if (emulator != null) bottomBarHeight else Dimens.TerminalKeyBarHeigh
    }
    if (showHostPicker) {
        HostPickerDialog(
            onDismiss = { showHostPicker = false },
            onPick = { hid ->
                showHostPicker = false
                TerminalSessionPool.ensureSession(hid)
            },
        )
    }
    if (showMoreMenu && active != null) {
        AlertDialog(
            onDismissRequest = { showMoreMenu = false },
            confirmButton = {
                TextButton(onClick = { showMoreMenu = false }) { Text(" 关闭") }
            },
            title = { Text(" 会话管理") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    TextButton(onClick = {
                        showMoreMenu = false
                        showHostPicker = true
                    }) { Text(" 新建会话") }
                    TextButton(onClick = {
                        showMoreMenu = false
                        TerminalSessionPool.closeOthers(active.sessionId)
                    }) { Text(" 关闭其它会话") }
                    TextButton(onClick = {
                        showMoreMenu = false
                        TerminalSessionPool.closeAll()
                    }) { Text(" 关闭全部会话") }
                    TextButton(onClick = {
                        showMoreMenu = false
                        TerminalSessionPool.broadcastWrite("\u0003")
```

```
                        }) { Text(" 广播 Ctrl+C") }
                    }
                },
            )
        }
    }
}
@Composable
private fun HostPickerDialog(
    onDismiss: () -> Unit,
    onPick: (Long) -> Unit,
) {
    val context = LocalContext.current
    val app = context.applicationContext as Application
    val repo = remember { HostRepository(DbProvider.get(app).hostDao()) }
    val hosts by repo.observeAll().map { it }.collectAsState(initial = emptyList())
    AlertDialog(
        onDismissRequest = onDismiss,
        confirmButton = {
            TextButton(onClick = onDismiss) { Text(" 关闭") }
        },
        title = { Text(" 新建会话") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                if (hosts.isEmpty()) {
                    Text(" 还没有主机，请先去「主机」页添加")
                } else {
                    hosts.forEach { h ->
                        Row(
                            modifier = Modifier
                                .fillMaxSize()
                                .clickable { onPick(h.id) }
                                .padding(vertical = 8.dp),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically,
                        ) {
                            Column {
                                Text(h.name)
                                Text("${h.username}@${h.host}:${h.port}", color = MaterialTheme.colorScheme.onSurfa
                            }
                            Icon(imageVector = Icons.Outlined.Add, contentDescription = "pick")
                        }
                    }
                }
            }
        },
    )
}
package com.dxkj.myshell.ui.screens
import android.app.Application
```

```
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import com.dxkj.myshell.ui.theme.Dimens
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.lifecycle.viewModelScope
import com.dxkj.myshell.data.db.DbProvider
import com.dxkj.myshell.data.db.HostEntity
import com.dxkj.myshell.data.repo.HostRepository
import com.dxkj.myshell.data.repo.KeyRepository
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.stateIn
import android.content.Context
@Composable
fun TerminalScreen(
    contentPadding: PaddingValues,
    onOpenFullTerminal: (Long) -> Unit,
) {
    val cfg = LocalConfiguration.current
    val isLandscape = cfg.screenWidthDp > cfg.screenHeightDp
    val context = LocalContext.current
    val prefs = remember(context) { context.getSharedPreferences("terminal_prefs", Context.MODE_PRIVATE) }
    val vm: TerminalViewModel = viewModel(factory = TerminalViewModel.factory(context.applicationContext as Applica
    val hosts by vm.hosts.collectAsState()
```

```
    val ui by vm.ui.collectAsState()
    var selectedHostId by remember { mutableStateOf<Long?>(null) }
    val lastHostId = remember { prefs.getLong("lastHostId", -1L).takeIf { it > 0 } }
    val recentIds = remember {
        (prefs.getString("recentHostIds", "") ?: "")
            .split(',')
            .mapNotNull { it.trim().takeIf { s -> s.isNotEmpty() }?.toLongOrNull() }
            .filter { it > 0 }
    }
    LaunchedEffect(hosts.size) {
        if (selectedHostId == null && lastHostId != null && hosts.any { it.id == lastHostId }) {
            selectedHostId = lastHostId
        }
    }
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(contentPadding)
            .padding(
                horizontal = Dimens.ScreenPaddingH,
                vertical = if (isLandscape) 12.dp else Dimens.ScreenPaddingV,
            ),
        verticalArrangement = Arrangement.spacedBy(if (isLandscape) 10.dp else Dimens.SpacingMd),
    ) {
        Text(text = " 终端", style = MaterialTheme.typography.titleLarge)
        if (hosts.isEmpty()) {
            Text(" 请先在「主机」页添加一个主机")
            return@Column
        }
        Text(" 选择主机: ")
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = Dimens.Spacing2),
            verticalArrangement = Arrangement.spacedBy(Dimens.SpacingXs),
        ) {
            items(hosts, key = { it.id }) { h ->
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(Dimens.SpacingSm, Alignment.Start),
                    verticalAlignment = Alignment.CenterVertically,
                ) {
                    Button(
                        onClick = { selectedHostId = h.id },
                    ) {
                        Text(if (selectedHostId == h.id) " 已选" else " 选择")
                    }
                    Column {
                        Text(h.name, style = MaterialTheme.typography.titleMedium)
                        Text("${h.username}@${h.host}:${h.port}", color = MaterialTheme.colorScheme.onSurfaceVarian
```

```
                }
            }
        }
    }
    Row(horizontalArrangement = Arrangement.spacedBy(Dimens.SpacingMd)) {
        Button(
            onClick = {
                val id = selectedHostId ?: return@Button
                onOpenFullTerminal(id)
            },
            enabled = selectedHostId != null,
        ) { Text(" 打开全屏终端") }
        if (lastHostId != null && selectedHostId != lastHostId) {
            Button(
                onClick = { onOpenFullTerminal(lastHostId) },
                enabled = hosts.any { it.id == lastHostId },
            ) { Text(" 继续上次会话") }
        }
    }
    if (recentIds.isNotEmpty()) {
        Text(" 最近会话: ", style = MaterialTheme.typography.titleMedium)
        val recentHosts = remember(hosts, recentIds) {
            val map = hosts.associateBy { it.id }
            recentIds.mapNotNull { map[it] }
        }
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = Dimens.Spacing2),
            verticalArrangement = Arrangement.spacedBy(Dimens.SpacingXs),
        ) {
            items(recentHosts, key = { it.id }) { h ->
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(Dimens.SpacingSm, Alignment.Start),
                    verticalAlignment = Alignment.CenterVertically,
                ) {
                    Button(onClick = { onOpenFullTerminal(h.id) }) { Text(" 打开") }
                    Column {
                        Text(h.name, style = MaterialTheme.typography.titleMedium)
                        Text("${h.username}@${h.host}:${h.port}", color = MaterialTheme.colorScheme.onSurfaceVa
                    }
                }
            }
        }
    }
    if (ui.status != null) {
        Text(ui.status ?: "", color = if (ui.statusOk) MaterialTheme.colorScheme.primary else MaterialTheme.col
    }
}
```

```
}
data class TerminalUi(
    val connecting: Boolean = false,
    val connected: Boolean = false,
    val status: String? = null,
    val statusOk: Boolean = false,
)
class TerminalViewModel(app: Application) : AndroidViewModel(app) {
    private val db = DbProvider.get(app)
    private val hostRepo = HostRepository(db.hostDao())
    val hosts: StateFlow<List<HostEntity>> =
        hostRepo.observeAll().stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
    private val _ui = kotlinx.coroutines.flow.MutableStateFlow(TerminalUi())
    val ui: StateFlow<TerminalUi> = _ui
    companion object {
        fun factory(app: Application): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    return TerminalViewModel(app) as T
                }
            }
    }
}
package com.dxkj.myshell.ui.terminal
import android.app.Activity
import android.content.Context
import android.view.HapticFeedbackConstants
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.isImeVisible
import androidx.compose.foundation.layout.navigationBars
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
```

```
import androidx.compose.material.icons.filled.ContentCut
import androidx.compose.material.icons.filled.Keyboard
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.FilledTonalButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.ExperimentalComposeUiApi
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.FocusRequester
import androidx.compose.ui.input.pointer.pointerInteropFilter
import androidx.compose.ui.platform.LocalClipboardManager
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalView
import androidx.compose.ui.text.AnnotatedString
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsCompat
import com.dxkj.myshell.ui.theme.Dimens
import org.json.JSONArray
import org.json.JSONObject
import androidx.compose.foundation.layout.windowInsetsPadding
import kotlin.math.roundToInt
private const val ESC = "\u001b"
private val KEY_ESC = byteArrayOf(0x1b)
private val KEY_TAB = byteArrayOf(0x09)
private val KEY_SHIFT_TAB = "$ESC[Z".toByteArray()
private const val VTERM_KEY_UP = 5
private const val VTERM_KEY_DOWN = 6
private const val VTERM_KEY_LEFT = 7
private const val VTERM_KEY_RIGHT = 8
private const val VTERM_KEY_HOME = 11
private const val VTERM_KEY_END = 12
private const val VTERM_KEY_PAGEUP = 13
private const val VTERM_KEY_PAGEDOWN = 14
private const val REPEAT_DELAY_MS = 400L
private const val REPEAT_INTERVAL_MS = 80L
```

```
private val NAV_CELL_WIDTH = 44.dp
private data class CustomKey(val label: String, val send: String)
@OptIn(ExperimentalLayoutApi::class)
@Composable
fun HavenKeyboardToolbar(
    focusRequester: FocusRequester,
    onSendBytes: (ByteArray) -> Unit,
    onDispatchKey: (modifiers: Int, key: Int) -> Unit,
    modifier: Modifier = Modifier,
    modifierManager: SimpleModifierManager,
) {
    val context = LocalContext.current
    val view = LocalView.current
    val imeVisible = WindowInsets.isImeVisible
    val clipboard = LocalClipboardManager.current
    val shiftActive = modifierManager.isShiftActive()
    val ctrlActive = modifierManager.isCtrlActive()
    val altActive = modifierManager.isAltActive()
    var showAddDialog by remember { mutableStateOf(false) }
    var showSnippets by remember { mutableStateOf(false) }
    val customKeys = remember { mutableStateListOf<CustomKey>() }
    LaunchedEffect(Unit) {
        customKeys.clear()
        customKeys.addAll(loadCustomKeys(context))
    }
    fun persist() {
        saveCustomKeys(context, customKeys.toList())
    }
    fun sendChar(ch: Char) {
        focusRequester.requestFocus()
        val b = if (ctrlActive && ch.code in 0x40..0x7F) {
            byteArrayOf((ch.code and 0x1F).toByte())
        } else {
            ch.toString().toByteArray()
        }
        if (altActive) {
            onSendBytes(byteArrayOf(0x1b) + b)
        } else {
            onSendBytes(b)
        }
        modifierManager.clearTransients()
    }
    fun paste() {
        focusRequester.requestFocus()
        val t = clipboard.getText()?.text?.toString().orEmpty()
        if (t.isNotEmpty()) onSendBytes(t.toByteArray())
        modifierManager.clearTransients()
    }
    fun sendBytesFromToolbar(bytes: ByteArray) {
        focusRequester.requestFocus()
```

```
        onSendBytes(bytes)
        modifierManager.clearTransients()
    }
    fun dispatchKeyFromToolbar(key: Int) {
        focusRequester.requestFocus()
        onDispatchKey(0, key)
        modifierManager.clearTransients()
    }
    if (showAddDialog) {
        AddCustomKeyDialog(
            onDismiss = { showAddDialog = false },
            onSave = { label, send ->
                customKeys.add(CustomKey(label.trim(), send))
                persist()
                showAddDialog = false
            },
        )
    }
    if (showSnippets) {
        SnippetsDialog(
            items = customKeys.toList(),
            onDismiss = { showSnippets = false },
            onTap = { k ->
                val send = k.send
                if (send == "PASTE") paste() else onSendBytes(send.toByteArray())
                showSnippets = false
            },
            onDelete = { k ->
                customKeys.removeAll { it.label == k.label && it.send == k.send }
                persist()
            },
            onAdd = { showSnippets = false; showAddDialog = true },
        )
    }
    Surface(
        tonalElevation = 2.dp,
        modifier = modifier
            .windowInsetsPadding(WindowInsets.navigationBars)
            .height(Dimens.TerminalKeyBarHeight)
            .fillMaxWidth(),
    ) {
        val scroll = rememberScrollState()
        Row(
            modifier = Modifier
                .horizontalScroll(scroll)
                .padding(horizontal = 6.dp, vertical = 6.dp),
            horizontalArrangement = Arrangement.spacedBy(6.dp),
        ) {
            Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
            ToolbarIconButton(
                icon = Icons.Filled.ContentCut,
                desc = "Snippets",
                onClick = { showSnippets = true },
                onLongClick = { view.performHapticFeedback(HapticFeedbackConstants.LONG_PRESS) },
            )
            ToolbarIconButton(
                icon = Icons.Filled.Keyboard,
                desc = "Keyboard",
                onClick = {
                    val window = (view.context as? Activity)?.window ?: return@ToolbarIconButton
                    val controller = WindowCompat.getInsetsController(window, view)
                    if (imeVisible) controller.hide(WindowInsetsCompat.Type.ime())
                    else {
                        focusRequester.requestFocus()
                        controller.show(WindowInsetsCompat.Type.ime())
                    }
                },
                onLongClick = { view.performHapticFeedback(HapticFeedbackConstants.LONG_PRESS) },
            )
            TextKey("Esc") { sendBytesFromToolbar(KEY_ESC) }
            TextKey("Tab") {
                if (shiftActive) {
                    sendBytesFromToolbar(KEY_SHIFT_TAB)
                } else {
                    sendBytesFromToolbar(KEY_TAB)
                }
            }
            TextKey("Paste") { paste() }
            SymbolKey("/") { sendChar('/') }
        }
        Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
            ToggleKey("Shift", shiftActive) { modifierManager.toggleShift() }
            ToggleKey("Ctrl", ctrlActive) { modifierManager.toggleCtrl() }
            ToggleKey("Alt", altActive) { modifierManager.toggleAlt() }
            TextKey("⌧") { sendBytesFromToolbar(byteArrayOf(0x7f.toByte())) }
            TextKey("End") { dispatchKeyFromToolbar(VTERM_KEY_END) }
        }
    }
    Column(
        modifier = Modifier
            .border(
                width = 1.dp,
                color = MaterialTheme.colorScheme.outline.copy(alpha = 0.25f),
                shape = RoundedCornerShape(12.dp),
            )
            .padding(horizontal = 2.dp, vertical = 2.dp),
        verticalArrangement = Arrangement.spacedBy(2.dp),
    ) {
        Row(horizontalArrangement = Arrangement.spacedBy(2.dp)) {
```

```
                    NavText("Home") { dispatchKeyFromToolbar(VTERM_KEY_HOME) }
                    NavArrow("↑") { dispatchKeyFromToolbar(VTERM_KEY_UP) }
                    NavText("End") { dispatchKeyFromToolbar(VTERM_KEY_END) }
                    NavText("PgUp") { dispatchKeyFromToolbar(VTERM_KEY_PAGEUP) }
                }
                Row(horizontalArrangement = Arrangement.spacedBy(2.dp)) {
                    NavArrow("←") { dispatchKeyFromToolbar(VTERM_KEY_LEFT) }
                    NavArrow("↓") { dispatchKeyFromToolbar(VTERM_KEY_DOWN) }
                    NavArrow("→") { dispatchKeyFromToolbar(VTERM_KEY_RIGHT) }
                    NavText("PgDn") { dispatchKeyFromToolbar(VTERM_KEY_PAGEDOWN) }
                }
            }
            Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    val row1 = customKeys.take(6)
                    row1.forEach { k ->
                        TextKey(k.label) {
                            if (k.send == "PASTE") paste() else sendBytesFromToolbar(k.send.toByteArray())
                        }
                    }
                }
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    val row2 = customKeys.drop(6).take(6)
                    row2.forEach { k ->
                        TextKey(k.label) {
                            if (k.send == "PASTE") paste() else sendBytesFromToolbar(k.send.toByteArray())
                        }
                    }
                    IconButton(
                        onClick = { showAddDialog = true },
                        modifier = Modifier.size(32.dp),
                    ) {
                        Icon(Icons.Filled.Add, contentDescription = "Add key", modifier = Modifier.size(18.dp))
                    }
                }
            }
        }
    }
}
@Composable
private fun NavCell(content: @Composable () -> Unit) {
    Box(
        modifier = Modifier.width(NAV_CELL_WIDTH).height(32.dp),
        contentAlignment = androidx.compose.ui.Alignment.Center,
    ) { content() }
}
@Composable
private fun NavArrow(label: String, onClick: () -> Unit) {
    NavCell {
        RepeatingButton(
```

```
            onClick = onClick,
            modifier = Modifier.fillMaxWidth().height(32.dp),
            contentPadding = PaddingValues(0.dp),
        ) {
            Text(label, fontSize = 16.sp, lineHeight = 16.sp)
        }
    }
}
@Composable
private fun NavText(label: String, onClick: () -> Unit) {
    NavCell {
        RepeatingButton(
            onClick = onClick,
            modifier = Modifier.fillMaxWidth().height(32.dp),
            contentPadding = PaddingValues(0.dp),
        ) {
            Text(label, fontSize = 11.sp, lineHeight = 11.sp)
        }
    }
}
@Composable
private fun TextKey(label: String, onClick: () -> Unit) {
    RepeatingButton(
        onClick = onClick,
        modifier = Modifier.height(32.dp),
        contentPadding = PaddingValues(horizontal = 8.dp, vertical = 0.dp),
    ) {
        Text(label, fontSize = 11.sp, lineHeight = 11.sp)
    }
}
@Composable
private fun SymbolKey(label: String, onClick: () -> Unit) = TextKey(label, onClick)
@Composable
private fun ToggleKey(label: String, active: Boolean, onClick: () -> Unit) {
    FilledTonalButton(
        onClick = onClick,
        modifier = Modifier.height(32.dp),
        contentPadding = PaddingValues(horizontal = 8.dp, vertical = 0.dp),
        colors = if (active) {
            ButtonDefaults.filledTonalButtonColors(
                containerColor = MaterialTheme.colorScheme.primary,
                contentColor = MaterialTheme.colorScheme.onPrimary,
            )
        } else ButtonDefaults.filledTonalButtonColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
        ),
    ) {
        Text(label, fontSize = 11.sp, lineHeight = 11.sp)
    }
}
```

```
@Composable
private fun ToolbarIconButton(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    desc: String,
    onClick: () -> Unit,
    onLongClick: () -> Unit,
    enabled: Boolean = true,
) {
    RepeatingButton(
        onClick = onClick,
        modifier = Modifier.size(32.dp),
        contentPadding = PaddingValues(0.dp),
        allowRepeat = false,
        onLongPress = onLongClick,
        enabled = enabled,
    ) {
        Icon(icon, contentDescription = desc, modifier = Modifier.size(18.dp))
    }
}
@OptIn(ExperimentalComposeUiApi::class)
@Composable
private fun RepeatingButton(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    contentPadding: PaddingValues = PaddingValues(horizontal = 10.dp, vertical = 0.dp),
    allowRepeat: Boolean = true,
    onLongPress: (() -> Unit)? = null,
    enabled: Boolean = true,
    content: @Composable () -> Unit,
) {
    var isPressed by remember { mutableStateOf(false) }
    var didRepeat by remember { mutableStateOf(false) }
    var downTime by remember { mutableStateOf(0L) }
    LaunchedEffect(isPressed, allowRepeat, enabled) {
        if (enabled && isPressed && allowRepeat) {
            didRepeat = false
            kotlinx.coroutines.delay(REPEAT_DELAY_MS)
            didRepeat = true
            while (true) {
                onClick()
                kotlinx.coroutines.delay(REPEAT_INTERVAL_MS)
            }
        }
    }
    FilledTonalButton(
        onClick = {},
        enabled = enabled,
        modifier = modifier.pointerInteropFilter { ev ->
            if (!enabled) return@pointerInteropFilter false
            when (ev.action) {
```

```
                android.view.MotionEvent.ACTION_DOWN -> {
                    isPressed = true
                    didRepeat = false
                    downTime = android.os.SystemClock.elapsedRealtime()
                    true
                }
                android.view.MotionEvent.ACTION_MOVE -> {
                    true
                }
                android.view.MotionEvent.ACTION_UP -> {
                    val elapsed = android.os.SystemClock.elapsedRealtime() - downTime
                    val longPress = elapsed >= 450L
                    if (longPress) onLongPress?.invoke()
                    else if (!didRepeat) onClick()
                    isPressed = false
                    true
                }
                android.view.MotionEvent.ACTION_CANCEL -> {
                    isPressed = false
                    true
                }
                else -> true
            }
        },
        contentPadding = contentPadding,
        colors = ButtonDefaults.filledTonalButtonColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
        ),
    ) { content() }
}
@Composable
private fun AddCustomKeyDialog(
    onDismiss: () -> Unit,
    onSave: (label: String, send: String) -> Unit,
) {
    var label by remember { mutableStateOf("") }
    var sendText by remember { mutableStateOf("") }
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(" 添加自定义键") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                OutlinedTextField(
                    value = label,
                    onValueChange = { label = it },
                    singleLine = true,
                    label = { Text(" 显示文字") },
                    placeholder = { Text(" 例如: ^C / Paste") },
                    modifier = Modifier.fillMaxWidth(),
                )
```

```
                OutlinedTextField(
                    value = sendText,
                    onValueChange = { sendText = it },
                    singleLine = true,
                    label = { Text(" 发送内容") },
                    placeholder = { Text(" 例如: \\u0003 或 PASTE 或 /") },
                    modifier = Modifier.fillMaxWidth(),
                )
                Text(
                    " 支持: \\u001b(Esc) \\u0003(Ctrl+C) \\n \\r \\t; 输入 PASTE 表示粘贴剪贴板。",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                )
            }
        },
        confirmButton = {
            TextButton(
                enabled = label.trim().isNotEmpty() && sendText.trim().isNotEmpty(),
                onClick = { onSave(label, parseSendSequence(sendText.trim())) },
            ) { Text(" 添加") }
        },
        dismissButton = { TextButton(onClick = onDismiss) { Text(" 取消") } },
    )
}
@Composable
private fun SnippetsDialog(
    items: List<CustomKey>,
    onDismiss: () -> Unit,
    onTap: (CustomKey) -> Unit,
    onDelete: (CustomKey) -> Unit,
    onAdd: () -> Unit,
) {
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(" 剪刀（Snippets）") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                if (items.isEmpty()) {
                    Text(" 还没有自定义键。你可以先点「添加」创建一个。", color = MaterialTheme.colorScheme.onSu
                } else {
                    items.forEach { k ->
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .background(MaterialTheme.colorScheme.surfaceVariant, RoundedCornerShape(10.dp))
                                .padding(horizontal = 10.dp, vertical = 8.dp),
                            horizontalArrangement = Arrangement.SpaceBetween,
                        ) {
                            Column(modifier = Modifier.weight(1f)) {
                                Text(k.label)
```

```
                                Text(
                                    displaySendSequence(k.send),
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                                    maxLines = 1,
                                )
                            }
                            Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                                TextButton(onClick = { onTap(k) }) { Text(" 发送") }
                                TextButton(onClick = { onDelete(k) }) { Text(" 删除") }
                            }
                        }
                    }
                }
                Spacer(modifier = Modifier.height(2.dp))
                FilledTonalButton(onClick = onAdd, modifier = Modifier.fillMaxWidth()) {
                    Text(" 添加自定义键")
                }
            }
        },
        confirmButton = { TextButton(onClick = onDismiss) { Text(" 关闭") } },
    )
}
private fun prefKey(): String = "terminal_haven_toolbar_custom_keys_v1"
private fun loadCustomKeys(context: Context): List<CustomKey> {
    val prefs = context.getSharedPreferences("terminal_prefs", Context.MODE_PRIVATE)
    val raw = prefs.getString(prefKey(), null) ?: return emptyList()
    return try {
        val arr = JSONArray(raw)
        buildList {
            for (i in 0 until arr.length()) {
                val o = arr.optJSONObject(i) ?: continue
                val label = o.optString("label").orEmpty()
                val send = o.optString("send").orEmpty()
                if (label.isNotBlank() && send.isNotBlank()) add(CustomKey(label, send))
            }
        }
    } catch (_: Throwable) {
        emptyList()
    }
}
private fun saveCustomKeys(context: Context, keys: List<CustomKey>) {
    val prefs = context.getSharedPreferences("terminal_prefs", Context.MODE_PRIVATE)
    val arr = JSONArray()
    keys.forEach { k ->
        arr.put(
            JSONObject()
                .put("label", k.label)
                .put("send", k.send),
        )
```

```
    }
    prefs.edit().putString(prefKey(), arr.toString()).apply()
}
private fun displaySendSequence(send: String): String {
    if (send == "PASTE") return "Paste clipboard"
    return send.map { ch ->
        when {
            ch.code < 0x20 -> "\\u${ch.code.toString(16).padStart(4, '0')}"
            else -> ch.toString()
        }
    }.joinToString("")
}
private fun parseSendSequence(input: String): String {
    if (input.equals("PASTE", ignoreCase = true)) return "PASTE"
    val sb = StringBuilder()
    var i = 0
    while (i < input.length) {
        if (i + 1 < input.length && input[i] == '\\') {
            when (input[i + 1]) {
                'n' -> { sb.append('\n'); i += 2 }
                't' -> { sb.append('\t'); i += 2 }
                'r' -> { sb.append('\r'); i += 2 }
                '\\' -> { sb.append('\\'); i += 2 }
                'u' -> {
                    if (i + 5 < input.length) {
                        val hex = input.substring(i + 2, i + 6)
                        val code = hex.toIntOrNull(16)
                        if (code != null) { sb.append(code.toChar()); i += 6 }
                        else { sb.append(input[i]); i++ }
                    } else { sb.append(input[i]); i++ }
                }
                else -> { sb.append(input[i]); i++ }
            }
        } else {
            sb.append(input[i])
            i++
        }
    }
    return sb.toString()
}
package com.dxkj.myshell.ui.terminal
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import org.connectbot.terminal.ModifierManager
class SimpleModifierManager : ModifierManager {
    private var ctrlLocked by mutableStateOf(false)
    private var altLocked by mutableStateOf(false)
    private var shiftTransient by mutableStateOf(false)
    fun toggleCtrl() { ctrlLocked = !ctrlLocked }
```

```
    fun toggleAlt() { altLocked = !altLocked }
    fun toggleShift() { shiftTransient = !shiftTransient }
    fun setCtrl(v: Boolean) { ctrlLocked = v }
    fun setAlt(v: Boolean) { altLocked = v }
    fun setShift(v: Boolean) { shiftTransient = v }
    override fun isCtrlActive(): Boolean = ctrlLocked
    override fun isAltActive(): Boolean = altLocked
    override fun isShiftActive(): Boolean = shiftTransient
    override fun clearTransients() {
        shiftTransient = false
    }
}
package com.dxkj.myshell.ui.terminal
import android.app.Activity
import android.view.HapticFeedbackConstants
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.isImeVisible
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.rememberScrollState
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Keyboard
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.FilledTonalButton
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.ExperimentalComposeUiApi
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.FocusRequester
import androidx.compose.ui.input.pointer.pointerInteropFilter
import androidx.compose.ui.platform.LocalClipboardManager
import androidx.compose.ui.platform.LocalView
import androidx.compose.ui.text.AnnotatedString
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsCompat
import com.dxkj.myshell.ui.theme.Dimens
```

```
private const val VTERM_KEY_UP = 5
private const val VTERM_KEY_DOWN = 6
private const val VTERM_KEY_LEFT = 7
private const val VTERM_KEY_RIGHT = 8
private const val VTERM_KEY_HOME = 11
private const val VTERM_KEY_END = 12
private const val VTERM_KEY_PAGEUP = 13
private const val VTERM_KEY_PAGEDOWN = 14
private const val REPEAT_DELAY_MS = 400L
private const val REPEAT_INTERVAL_MS = 80L
@Composable
@OptIn(ExperimentalLayoutApi::class)
fun TerminalKeyboardToolbar(
    focusRequester: FocusRequester,
    onSendBytes: (ByteArray) -> Unit,
    onDispatchKey: (modifiers: Int, key: Int) -> Unit,
    modifier: Modifier = Modifier,
) {
    val view = LocalView.current
    val imeVisible = WindowInsets.isImeVisible
    val clipboard = LocalClipboardManager.current
    Surface(
        tonalElevation = 2.dp,
        modifier = modifier.height(Dimens.TerminalKeyBarHeight),
    ) {
        Row(
            modifier = Modifier
                .horizontalScroll(rememberScrollState())
                .padding(horizontal = 6.dp, vertical = 6.dp),
        ) {
            ToolbarIconButton(
                onClick = {
                    val window = (view.context as? Activity)?.window ?: return@ToolbarIconButton
                    val controller = WindowCompat.getInsetsController(window, view)
                    if (imeVisible) {
                        controller.hide(WindowInsetsCompat.Type.ime())
                    } else {
                        focusRequester.requestFocus()
                        controller.show(WindowInsetsCompat.Type.ime())
                    }
                },
                onLongClick = {
                    view.performHapticFeedback(HapticFeedbackConstants.LONG_PRESS)
                },
            )
            SpacerKey()
            ToolbarTextKey("Esc") { onSendBytes(byteArrayOf(0x1b)) }
            ToolbarTextKey("Tab") { onSendBytes(byteArrayOf(0x09)) }
            ToolbarTextKey("Enter") { onSendBytes(byteArrayOf('\r'.code.toByte())) }
            ToolbarTextKey("⌧") { onSendBytes(byteArrayOf(0x7f.toByte())) }
```

```
            ToolbarTextKey("Ctrl+C") { onSendBytes(byteArrayOf(0x03)) }
            SpacerKey()
            ToolbarArrowKey("↑") { onDispatchKey(0, VTERM_KEY_UP) }
            ToolbarArrowKey("↓") { onDispatchKey(0, VTERM_KEY_DOWN) }
            ToolbarArrowKey("←") { onDispatchKey(0, VTERM_KEY_LEFT) }
            ToolbarArrowKey("→") { onDispatchKey(0, VTERM_KEY_RIGHT) }
            ToolbarTextKey("Home") { onDispatchKey(0, VTERM_KEY_HOME) }
            ToolbarTextKey("End") { onDispatchKey(0, VTERM_KEY_END) }
            ToolbarTextKey("PgUp") { onDispatchKey(0, VTERM_KEY_PAGEUP) }
            ToolbarTextKey("PgDn") { onDispatchKey(0, VTERM_KEY_PAGEDOWN) }
            SpacerKey()
            ToolbarTextKey("Paste") {
                val t = clipboard.getText()?.text?.toString().orEmpty()
                if (t.isNotEmpty()) onSendBytes(t.toByteArray())
            }
            ToolbarTextKey("Copy") {
            }
        }
    }
}
@Composable
private fun SpacerKey() {
    androidx.compose.foundation.layout.Spacer(modifier = Modifier.size(6.dp))
}
@Composable
private fun ToolbarIconButton(
    onClick: () -> Unit,
    onLongClick: () -> Unit,
) {
    RepeatingButton(
        onClick = onClick,
        modifier = Modifier.size(36.dp),
        contentPadding = PaddingValues(0.dp),
        allowRepeat = false,
        onLongPress = onLongClick,
    ) {
        Icon(Icons.Filled.Keyboard, contentDescription = "Toggle keyboard", modifier = Modifier.size(18.dp))
    }
}
@Composable
private fun ToolbarTextKey(label: String, onClick: () -> Unit) {
    RepeatingButton(
        onClick = onClick,
        modifier = Modifier
            .padding(horizontal = 2.dp)
            .height(32.dp),
    ) {
        androidx.compose.material3.Text(label, fontSize = 11.sp, lineHeight = 11.sp)
    }
}
```

```
@Composable
private fun ToolbarArrowKey(label: String, onClick: () -> Unit) {
    RepeatingButton(
        onClick = onClick,
        modifier = Modifier
            .padding(horizontal = 2.dp)
            .height(32.dp),
    ) {
        androidx.compose.material3.Text(label, fontSize = 16.sp, lineHeight = 16.sp)
    }
}
@OptIn(ExperimentalComposeUiApi::class)
@Composable
private fun RepeatingButton(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    contentPadding: PaddingValues = PaddingValues(horizontal = 10.dp, vertical = 0.dp),
    allowRepeat: Boolean = true,
    onLongPress: (() -> Unit)? = null,
    content: @Composable () -> Unit,
) {
    var isPressed by remember { mutableStateOf(false) }
    var didRepeat by remember { mutableStateOf(false) }
    var downTime by remember { mutableStateOf(0L) }
    LaunchedEffect(isPressed, allowRepeat) {
        if (isPressed && allowRepeat) {
            didRepeat = false
            kotlinx.coroutines.delay(REPEAT_DELAY_MS)
            didRepeat = true
            while (true) {
                onClick()
                kotlinx.coroutines.delay(REPEAT_INTERVAL_MS)
            }
        }
    }
    FilledTonalButton(
        onClick = {},
        modifier = modifier.pointerInteropFilter { ev ->
            when (ev.action) {
                android.view.MotionEvent.ACTION_DOWN -> {
                    isPressed = true
                    didRepeat = false
                    downTime = android.os.SystemClock.elapsedRealtime()
                    true
                }
                android.view.MotionEvent.ACTION_UP -> {
                    val elapsed = android.os.SystemClock.elapsedRealtime() - downTime
                    val longPress = elapsed >= 450L
                    if (longPress) {
                        onLongPress?.invoke()
```

```
                    } else if (!didRepeat) {
                        onClick()
                    }
                    isPressed = false
                    true
                }
                android.view.MotionEvent.ACTION_CANCEL -> {
                    isPressed = false
                    true
                }
                else -> false
            }
        },
        contentPadding = contentPadding,
        colors = ButtonDefaults.filledTonalButtonColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
        ),
    ) {
        content()
    }
}
package com.dxkj.myshell.ui.theme
import androidx.compose.ui.unit.dp
object Dimens {
    val ScreenPaddingH = 16.dp
    val ScreenPaddingV = 16.dp
    val CardPadding = 16.dp
    val SpacingXs = 6.dp
    val Spacing2 = 4.dp
    val Spacing1 = 2.dp
    val SpacingSm = 8.dp
    val SpacingMd = 12.dp
    val SpacingLg = 16.dp
    val SpacingXl = 24.dp
    val SidebarPadding = 10.dp
    val OverlayCorner = 14.dp
    val OverlayCornerSm = 12.dp
    val OverlayPaddingH = 12.dp
    val OverlayPaddingV = 8.dp
    val OverlayChipPaddingH = 10.dp
    val OverlayChipPaddingV = 6.dp
    val OverlayGap = 6.dp
    val TerminalKeyBarHeight = 72.dp
}
package com.dxkj.myshell.ui.theme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color
```

```
import androidx.compose.foundation.isSystemInDarkTheme
private val LightColors = lightColorScheme(
    primary = Color(0xFF19C37D),
    onPrimary = Color(0xFF062016),
    primaryContainer = Color(0xFFCFFAE6),
    onPrimaryContainer = Color(0xFF052014),
    secondary = Color(0xFF21B6C7),
    onSecondary = Color(0xFF001F23),
    secondaryContainer = Color(0xFFB7F2FA),
    onSecondaryContainer = Color(0xFF002022),
    tertiary = Color(0xFF94A3B8),
    onTertiary = Color(0xFF0B1220),
    tertiaryContainer = Color(0xFFE2E8F0),
    onTertiaryContainer = Color(0xFF0B1220),
    background = Color(0xFFF5F6F7),
    onBackground = Color(0xFF111827),
    surface = Color(0xFFFFFFFF),
    onSurface = Color(0xFF111827),
    surfaceVariant = Color(0xFFF0F2F4),
    onSurfaceVariant = Color(0xFF4B5563),
    outline = Color(0xFFD1D5DB),
    outlineVariant = Color(0xFFE5E7EB),
    error = Color(0xFFDC2626),
    onError = Color(0xFFFFFFFF),
    errorContainer = Color(0xFFFEE2E2),
    onErrorContainer = Color(0xFF450A0A),
)
private val DarkColors = darkColorScheme(
    primary = Color(0xFF19C37D),
    onPrimary = Color(0xFF062016),
    primaryContainer = Color(0xFF0C3B29),
    onPrimaryContainer = Color(0xFFCFFAE6),
    secondary = Color(0xFF21B6C7),
    onSecondary = Color(0xFF001F23),
    secondaryContainer = Color(0xFF0D3A40),
    onSecondaryContainer = Color(0xFFB7F2FA),
    tertiary = Color(0xFF94A3B8),
    onTertiary = Color(0xFF0B1220),
    tertiaryContainer = Color(0xFF1F2937),
    onTertiaryContainer = Color(0xFFE5E7EB),
    background = Color(0xFF0B0F14),
    onBackground = Color(0xFFE5E7EB),
    surface = Color(0xFF0F141B),
    onSurface = Color(0xFFE5E7EB),
    surfaceVariant = Color(0xFF151B23),
    onSurfaceVariant = Color(0xFFA3AAB6),
    outline = Color(0xFF2A3340),
    outlineVariant = Color(0xFF1B2430),
    error = Color(0xFFF87171),
    onError = Color(0xFF450A0A),
```

```
    errorContainer = Color(0xFF7F1D1D),
    onErrorContainer = Color(0xFFFEE2E2),
)
@Composable
fun MyShellTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    val colors = if (darkTheme) DarkColors else LightColors
    MaterialTheme(
        colorScheme = colors,
        typography = AppTypography,
        content = content,
    )
}
package com.dxkj.myshell.ui.theme
import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp
val AppTypography = Typography(
    titleLarge = TextStyle(
        fontSize = 22.sp,
        lineHeight = 28.sp,
        fontWeight = FontWeight.SemiBold,
    ),
    titleMedium = TextStyle(
        fontSize = 18.sp,
        lineHeight = 24.sp,
        fontWeight = FontWeight.SemiBold,
    ),
    bodyMedium = TextStyle(
        fontSize = 14.sp,
        lineHeight = 20.sp,
    ),
    bodySmall = TextStyle(
        fontSize = 12.sp,
        lineHeight = 16.sp,
    ),
    labelLarge = TextStyle(
        fontSize = 13.sp,
        lineHeight = 18.sp,
        fontWeight = FontWeight.Medium,
    ),
    labelSmall = TextStyle(
        fontSize = 11.sp,
        lineHeight = 14.sp,
        fontWeight = FontWeight.Medium,
    ),
)
```